

Tecnologie Web

Marco Tartaglione

7 ottobre 2026

Indice

1	URI	7
1.1	Uniformità	7
1.2	Sintassi	7
1.3	Specializzazioni	7
1.3.1	URN	7
1.3.2	URL	7
1.4	Opacità e gerarchie	8
1.4.1	Operazioni sulle URI gerarchiche	8
2	HTTP	9
2.1	Terminologia	9
2.2	Versioni	9
2.2.1	HTTP/1.1	9
2.2.2	HTTP/2	9
2.2.3	HTTP/3	10
2.3	Messaggi	10
2.3.1	Header	10
2.4	Comandi di richiesta	11
2.5	Risposta	11
2.5.1	Codici di stato	12
2.6	Cookies	12
2.6.1	Struttura	12
2.7	Autenticazione	12
2.8	SSL/TLS	13
2.9	Architetture	13
2.10	Caching	13
2.10.1	Meccanismi collaterali	14
3	Html	15
4	CSS	16
5	Web Dinamico	17
6	Servlet	18
6.1	Web Application	18
6.2	Servlet	18
6.2.1	Gerarchia	18
6.3	Request e Response	18
6.4	Ciclo di vita	18
6.5	Multithreading	18
6.5.1	Modello Single-Threaded	19
6.6	Metodi di controllo	19
6.7	Response	19
6.7.1	Status code	19
6.7.2	Header	20
6.7.3	Contenuto	20
6.8	Request	20
6.8.1	URL	20
6.8.2	Autenticazione, sicurezza e cookie	21
6.8.3	Contenuto	21
6.9	Deployment	21
6.9.1	Web Archives	21

6.9.2	Configurazione della Servlet	22
6.9.3	Contesto	22
6.10	Sessione	22
6.10.1	Cookie	23
6.10.2	Web Container	23
6.10.3	Accesso	23
6.10.4	Metodi	23
6.11	Scope differenziati	23
6.11.1	Funzionalità degli oggetti scoped	23
6.12	Inclusione di risorse	24
6.13	Inoltro	24
6.14	Redirezione	24
7	Java Server Pages	25
7.1	JSP Servlet	25
7.2	Differenza con le Servlet	25
7.2.1	Flusso	25
7.3	Tag	25
7.3.1	Scripting-oriented	25
7.3.2	XML-oriented	26
7.4	Direttive	26
7.4.1	Page	26
7.4.2	Include	27
7.4.3	Taglib	27
7.5	Built-in objects	27
7.6	Azioni	28
7.6.1	Forward	28
7.6.2	Include	28
7.7	Modello a componenti	29
7.7.1	Proprietà	29
7.7.2	Componenti	29
7.8	JSP e Bean	29
7.8.1	useBean	29
7.8.2	getProperty	30
7.8.3	setProperty	30
7.9	Model 1	30
7.10	Custom tag	30
7.10.1	Tag library	31
7.10.2	Implementazione tag	31
8	Javascript	32
8.1	Standardizzazione	32
8.2	Java	32
8.3	Utilizzi	32
8.4	Sintassi	32
8.4.1	Variabili	33
8.4.2	Valori speciali	33
8.4.3	Tipi primitivi	33
8.4.4	Literal	33
8.5	Oggetti	33
8.5.1	Costruzione	34
8.5.2	Literal	34
8.5.3	Array	34
8.6	Correlazione tra Oggetti e Array	34

8.7	Stringhe	35
8.7.1	Wrapper String	35
8.7.2	Espressioni regolari	36
8.8	Funzioni	36
8.8.1	Literal e costruttori	37
8.8.2	Parametri e argomenti	37
8.8.3	Funzioni anonime e arrow function	37
8.8.4	Metodi	37
8.8.5	Costruttori	38
8.8.6	Metodi statici	38
8.9	Operatori	39
8.10	Istruzioni	39
8.10.1	Istruzioni di controllo	39
8.11	Oggetto globale	40
8.12	JavaScript in HTML	40
8.12.1	Disattivazione di JavaScript	40
8.13	Utilizzi di JavaScript	41
8.14	Browser Objects	41
8.14.1	Costruzione dinamica della pagina	41
8.14.2	Rilevazione del browser	41
8.14.3	Rilevazione delle proprietà dello schermo	41
8.15	Modello ad eventi	42
8.15.1	Eventi	42
8.15.2	Handlers	42
8.16	Esplorazione del DOM	43
8.16.1	Oggetto document	43
8.16.2	Oggetto Form	43
8.17	Widget di un form	44
8.17.1	Proprietà e metodi dell'oggetto Text	44
8.17.2	Proprietà dell'oggetto Checkbox/Radio	44
8.18	Validazione del form	44
9	Ajax	46
9.1	Postback	46
9.1.1	Limiti	46
9.2	Nascita Ajax	46
9.2.1	Sequenza AJAX	47
9.3	XMLHttpRequest	47
9.3.1	Creazione	47
9.3.2	Metodi	47
9.3.3	Proprietà	48
9.3.4	Funzione di callback	49
9.4	Vantaggi e svantaggi	49
9.4.1	Vantaggi	49
9.4.2	Svantaggi	50
9.5	Criticità di comunicazione	50
9.5.1	Metodo <code>abort()</code>	50
9.6	Gestione della risposta	50
9.7	JSON	51
9.7.1	Da stringa ad oggetto	51
9.7.2	Java	51

10 React	52
10.1 Approccio	52
10.1.1 Virtual DOM	52
10.2 Vantaggi	52
10.3 Element	52
10.3.1 Esempio	53
10.4 JSX	53
10.4.1 Variabili	53
10.4.2 Costanti	54
10.4.3 Funzioni	54
10.4.4 Compilatore	54
10.5 Componenti	55
10.5.1 Tipi	55
10.5.2 Classi in linea	55
10.6 Props	55
10.7 State	56
10.7.1 Costruttore	56
10.7.2 Aggiornamento	56
10.7.3 Considerazioni	57
10.8 Gestione degli eventi	57
10.8.1 Eventi React	57
10.8.2 Accesso allo state	57
10.9 Form	58
10.9.1 Invocazione risorse server	58
10.10 Librerie alternative	59
10.10.1 JQuery	59
10.10.2 Angular	59
10.10.3 Vue	59
10.10.4 Meteor	59
10.10.5 Backbone	60
11 Tecnologie e componenti server-side	61
11.1 Java Model 2	61
11.2 Model-View-Controller	61
11.3 Delega al container	61
11.3.1 Container proprietari	61
11.3.2 Container standard-based	62
11.3.3 J2EE per applicazioni n-tier	62
11.4 EJB	62
11.4.1 Architettura	62
11.4.2 Descrittori	62
11.4.3 Componenti	62
11.4.4 Bean in EJB 3.0	63
11.5 Servizi container based	63
11.5.1 Gestione della concorrenza	63
11.5.2 Stateless Session Bean	63
11.5.3 Stateful Session Bean	64
11.5.4 Message-Driven Bean	64
11.6 Spring	64
11.6.1 Funzionalità chiave	64
11.6.2 Architettura	65
11.6.3 Dependency Injection	65
11.6.4 BeanFactory	65
11.6.5 Parametri di injection	66

11.6.6	Naming dei componenti	67
11.6.7	Supporto a MVC	67
11.6.8	Dispatcher servlet	67
11.6.9	Controller	68
11.6.10	Handler	68
11.6.11	View	68
11.6.12	Tag	69
11.6.13	Gestione delle eccezioni	69
11.6.14	Convenzioni e configurazione di default	69
12	Node.js e HTTP/3	70
12.1	Node.js	70
12.1.1	Event loop	70
12.1.2	Operazioni I/O sincrone	70
12.1.3	In a nutshell	70
12.1.4	Quando utilizzare Node.js	71
12.1.5	Server side e Client side	71
12.1.6	Moduli	71
12.1.7	Listener/Emitter Pattern	71
12.1.8	Stream	71
12.1.9	TCP Networking	72
12.2	HTTP/3	72
13	Websocket e JSF	73
13.1	JSF	73
13.1.1	Utilizzo	73
13.1.2	Ciclo di vita	73
13.1.3	Templating	73
13.1.4	faces-config.xml	74
13.2	WebSocket	74
13.2.1	Caratteristiche	75
13.2.2	Elementi base	75
13.2.3	Java API	75
13.2.4	Integrazione con JavaScript	77

1 URI

Gli **URI** (Uniform Resource Identifier) consentono di identificare una risorsa all'interno di una rete, ossia qualsiasi entità accessibile o riferibile.

Si tratta di un identificatore generico:

- non è vincolato al protocollo HTTP;
- non punta a una versione specifica della risorsa, ma al suo riferimento logico o al mapping della risorsa stessa.

1.1 Uniformità

Gli URI seguono una sintassi standard, garantendo uniformità nella loro rappresentazione. Questo comporta diversi vantaggi:

- semantica comune, condivisa tra tutti gli utenti e sistemi;
- convenzioni uniformi per la costruzione e interpretazione degli URI;
- facilità di introduzione di nuovi identificatori senza ambiguità;
- possibilità di riutilizzare lo stesso identificatore su protocolli diversi.

1.2 Sintassi

La sintassi generale di un URI è la seguente:

`<scheme>:<scheme-specific-part>`

La parte specifica del **scheme** non segue una struttura fissa, ma spesso assume la forma gerarchica:

`<scheme>://<authority><path>?<query>`

Questa struttura è particolarmente adatta a rappresentare relazioni gerarchiche all'interno di uno spazio di nomi.

1.3 Specializzazioni

Il concetto di URI si specializza poi in un'altra serie di concetti definiti

1.3.1 URN

Un *URN* (Uniform Resource Name) identifica una risorsa tramite un nome unico all'interno di uno specifico namespace. Questo permette di riferirsi a una risorsa indipendentemente dalla sua locazione o dal protocollo di accesso.

1.3.2 URL

Un *URL* (Uniform Resource Locator) identifica una risorsa specificando anche il meccanismo di accesso, ovvero il protocollo utilizzato (tipicamente HTTP, HTTPS, FTP, ecc.). La forma più comune è:

`<protocol>://[<username>:<password>@]<host>[:<port>][<path>[?<query>][<fragment>]]`

Dove:

- `<protocol>`: definisce il protocollo di accesso alla risorsa (es. http, https, ftp);
- `<username>:<password>@`: opzionale, indica le credenziali per l'accesso alla risorsa;
- `<host>`: nome del server o indirizzo IP che ospita la risorsa;
- `[:<port>]`: opzionale, porta di accesso al server (default 80 per HTTP, 443 per HTTPS);
- `[<path>]`: percorso della risorsa sul server;
- `[?<query>]`: opzionale, stringa di parametri chiave-valore passati al server;
- `[<fragment>]`: opzionale, riferimento a una sezione specifica della risorsa.

1.4 Opacità e gerarchie

Le URI possono essere classificate anche in base alla loro struttura:

- **Opaque:** non soggette a operazioni di parsing; la loro struttura interna non è interpretata.
- **Gerarchiche:** soggette a operazioni di parsing; la loro struttura interna (scheme, authority, path, query, fragment) è interpretabile.

1.4.1 Operazioni sulle URI gerarchiche

Sulle URI gerarchiche è possibile eseguire diverse operazioni:

- **Normalizzazione:** rimozione dei segmenti `.` e `..` per ottenere un percorso canonico.
- **Risoluzione:** partendo da una URI di base, calcola una URI risultante relativa a quella di riferimento.
- **Relativizzazione:** processo inverso alla risoluzione, ottenendo un percorso relativo rispetto a una URI di base.

2 HTTP

HTTP (HyperText Transfer Protocol) è il protocollo di livello applicativo utilizzato per il trasferimento di risorse sul web, basato sul meccanismo di *request-response*. Si tratta di un protocollo **stateless**, il che significa che né il client né il server mantengono, a livello di protocollo, informazioni sulle comunicazioni precedenti.

Il protocollo HTTP/1.0, in particolare, presenta le seguenti caratteristiche principali:

- **Stateless**: ogni richiesta è indipendente dalle precedenti.
- **One-shot**: ogni richiesta apre una nuova connessione TCP e viene chiusa subito dopo la risposta.
- **Request-Response**: il client invia una richiesta e attende la risposta del server.
- **Basato su TCP**: HTTP sfrutta le *stream* del protocollo TCP per garantire il trasporto affidabile dei messaggi.

2.1 Terminologia

Le principali entità e concetti di HTTP sono:

- **Client**: il soggetto che invia richieste al server per ottenere risorse.
- **Server**: il soggetto che riceve le richieste e fornisce le risposte.
- **Connessione**: il canale attraverso cui client e server comunicano; può essere *persistente* o *non persistente*.
- **Messaggio**: unità di comunicazione scambiata tra client e server.
 - **Request**: messaggio inviato dal client per richiedere una risorsa.
 - **Response**: messaggio inviato dal server in risposta alla richiesta del client.
- **Risorsa (Resource)**: qualsiasi contenuto o servizio accessibile tramite HTTP.
- **URI (Uniform Resource Identifier)**: identificatore univoco di una risorsa.
- **Entity**: rappresenta il contenuto effettivo della risorsa trasferita, comprensivo di header e body.

2.2 Versioni

2.2.1 HTTP/1.1

Con la versione 1.1, HTTP introduce il concetto di **connessioni persistenti**, permettendo di riutilizzare la stessa connessione TCP per più richieste e risposte tra client e server. A differenza di HTTP/1.0, in cui ogni richiesta apre e chiude una nuova connessione (connessione *non persistente*), le connessioni HTTP/1.1 rimangono aperte fino a quando il client non invia un messaggio di chiusura o fino a che non viene raggiunto un timeout per inattività.

Questo approccio riduce significativamente il *latency overhead* dovuto all'apertura e chiusura ripetuta di connessioni TCP, migliorando le prestazioni soprattutto per le applicazioni web moderne che richiedono più risorse per pagina.

Pipelining HTTP/1.1 introduce anche il **pipelining**, una funzionalità che consente al client di inviare più richieste consecutive senza attendere la risposta di ciascuna.

Le risposte del server devono essere restituite nello stesso ordine in cui le richieste sono state ricevute. Questo permette di ridurre i tempi di attesa rispetto al modello *stop-and-wait* di HTTP/1.0 ma richiede che il server supporti l'ordine delle risposte e la corretta sincronizzazione delle richieste.

2.2.2 HTTP/2

HTTP/2 è la revisione del protocollo HTTP pensata per migliorare le prestazioni e l'efficienza della comunicazione web, ed è basato su SPDY, un protocollo di *open networking* promosso da Google.

Le principali caratteristiche di HTTP/2 sono:

- **Multiplexing**: più richieste e risposte possono condividere la stessa connessione TCP senza bloccare l'una l'altra;

- **Header compression:** gli header HTTP vengono compressi (HPACK), riducendo l'overhead;
- **Server push:** il server può inviare risorse al client prima ancora che vengano richieste esplicitamente;
- **Prioritizzazione delle richieste:** permette al client di indicare l'importanza relativa delle richieste.

HTTP/2 mantiene la semantica di HTTP/1.1 (metodi, status code, URI, ecc.) ma migliora notevolmente l'efficienza della trasmissione dei dati.

2.2.3 HTTP/3

HTTP/3 è la più recente evoluzione del protocollo HTTP, sviluppata dal Working Group Hypertext Transfer Protocol dell'IETF e pubblicata come draft RFC 9114 a settembre 2022. HTTP/3 nasce per superare i limiti di TCP e HTTP/2, integrando QUIC, un protocollo basato su UDP che fornisce trasmissioni affidabili e sicure con minore latenza.

Le principali caratteristiche di HTTP/3 sono:

- **Trasporto su QUIC:** ogni connessione HTTP/3 è incapsulata in una connessione QUIC, che gestisce affidabilità, crittografia e controllo del flusso;
- **Zero head-of-line blocking:** a differenza di HTTP/2 su TCP, la perdita di un pacchetto non blocca le altre richieste della stessa connessione;
- **Handshake più rapido:** QUIC combina handshake crittografico e di trasporto, riducendo i tempi di connessione iniziale;
- **Multiplexing avanzato:** le richieste e risposte continuano ad essere multiplexate senza i limiti di TCP.

HTTP/3 mantiene la stessa semantica di HTTP tradizionale, ma migliora notevolmente la velocità, la resilienza e l'efficienza delle comunicazioni web.

2.3 Messaggi

I messaggi HTTP seguono una struttura ben definita, comune sia alle richieste (*request*) sia alle risposte (*response*). Ogni messaggio è composto da una parte di controllo e, opzionalmente, da una parte contenente i dati veri e propri.

I messaggi di **response** includono tipicamente la rappresentazione della risorsa richiesta dal client. I dati sono codificati secondo il formato indicato negli header (ad esempio tramite il campo **Content-Type**) e spesso utilizzano formati MIME come **text/html**, **application/json**, **image/png**, ecc.

In generale, un messaggio HTTP è composto da:

- **Header:** insieme di campi che descrivono il messaggio e forniscono informazioni di controllo, come il tipo di contenuto, la lunghezza del body, le direttive di caching, l'autenticazione e le caratteristiche della connessione;
- **Body:** parte opzionale che contiene i dati del messaggio, come il contenuto di una risorsa (nelle response) o i dati inviati dal client al server (nelle request, ad esempio in una POST).

Negli HTTP message, gli header sono separati dal body tramite una riga vuota, che indica la fine della sezione di intestazione e l'inizio del contenuto.

2.3.1 Header

Gli **header HTTP** sono costituiti da coppie *nome-valore* che descrivono le caratteristiche del messaggio e forniscono informazioni di controllo necessarie alla corretta gestione della comunicazione tra client e server. Ogni header specifica un particolare aspetto della richiesta o della risposta, come il formato dei dati, la lunghezza del contenuto, le politiche di caching, l'autenticazione o il tipo di connessione.

Gli header HTTP possono essere classificati nelle seguenti categorie:

- **Header generali:** validi sia per le richieste che per le risposte e relativi alla trasmissione nel suo complesso (ad esempio informazioni sulla connessione o sul caching);

- **Header di entità:** descrivono le caratteristiche del contenuto trasmesso, come il tipo di media, la codifica o la lunghezza dell'entità;
- **Header di richiesta:** presenti solo nei messaggi di request e forniscono informazioni aggiuntive sulla richiesta effettuata dal client, come le preferenze sul formato della risposta o i dati di autenticazione;
- **Header di risposta:** presenti solo nei messaggi di response e descrivono lo stato dell'operazione richiesta o le caratteristiche della risorsa restituita dal server.

2.4 Comandi di richiesta

I **comandi di richiesta** (o *metodi HTTP*) definiscono le operazioni che il client può richiedere al server tramite una richiesta HTTP. Ogni metodo specifica la semantica dell'azione da eseguire sulla risorsa identificata dall'URI.

I principali metodi HTTP sono:

- **GET:** richiede la rappresentazione di una risorsa. I parametri possono essere passati tramite la *query string* dell'URL, che è soggetta a limiti di lunghezza. Il metodo GET non dovrebbe produrre effetti collaterali sullo stato del server (operazione *safe*).
- **POST:** utilizzato per inviare dati dal client al server, tipicamente per l'elaborazione di una richiesta. I dati vengono trasmessi nel *body* del messaggio e non nell'URL. Non è soggetto a limiti pratici di lunghezza ed è spesso usato per operazioni che modificano lo stato del server senza creare direttamente una nuova risorsa.
- **PUT:** richiede la creazione o la sostituzione di una risorsa sul server all'URI specificato. Dopo un'operazione di PUT, una successiva richiesta GET sullo stesso URI dovrebbe restituire la risorsa appena memorizzata.
- **DELETE:** richiede la cancellazione di una risorsa identificata dall'URI. Per motivi di sicurezza, questo metodo è spesso disabilitato o limitato nei server pubblici.
- **HEAD:** equivalente a GET, ma il server restituisce esclusivamente gli *header* della risposta, senza il *body*. È utile per verificare l'esistenza o le caratteristiche di una risorsa.
- **OPTIONS:** consente di interrogare il server sulle opzioni di comunicazione supportate per una risorsa, come i metodi HTTP consentiti.
- **TRACE:** esegue un *loop-back* a livello applicativo, restituendo al client la richiesta ricevuta dal server. È utilizzato principalmente per scopi di diagnostica e debug.

2.5 Risposta

Una **risposta HTTP** è il messaggio inviato dal server al client in seguito alla ricezione e all'elaborazione di una richiesta. Essa è composta da una **riga di stato**, da un insieme di **header** e da un **body** opzionale contenente la rappresentazione della risorsa richiesta.

Un esempio di risposta HTTP è il seguente:

```
HTTP/1.1 200 OK
Connection: close
Date: Thu, 06 Aug 2008 12:00:15 GMT
Server: Apache/2.3.0 (Unix)
Last-Modified: Mon, 22 Jun 2008
Content-Length: 6821
Content-Type: text/html

<!DOCTYPE HTML PUBLIC "-//W3C//DTD
HTML 4.51 Transitional//EN">
<html>...</html>
```

2.5.1 Codici di stato

I **codici di stato HTTP** sono numeri a tre cifre inclusi nella risposta del server e indicano l'esito della richiesta effettuata dal client. La prima cifra identifica la **classe della risposta**, mentre le due cifre successive specificano il significato dettagliato del risultato.

Le principali classi di codici di stato sono:

- **1xx – Informational**: la richiesta è stata ricevuta e il processo è in corso;
- **2xx – Successful**: la richiesta è stata ricevuta, compresa ed elaborata con successo;
- **3xx – Redirection**: sono necessarie ulteriori azioni da parte del client per completare la richiesta (ad esempio una redirection);
- **4xx – Client Error**: la richiesta contiene un errore o non può essere soddisfatta dal server a causa del client;
- **5xx – Server Error**: il server non è riuscito a soddisfare una richiesta apparentemente valida.

2.6 Cookies

Oltre al meccanismo *request/response*, HTTP mette a disposizione una struttura per il salvataggio e la trasmissione di informazioni di stato: i **cookie**.

I cookie sono piccole coppie *chiave-valore* che possono essere **generati dal server** ed inviati al client tramite l'header **Set-Cookie**, oppure **creati dal client**. Una volta memorizzati dal browser, i cookie vengono automaticamente rinviati al server in ogni richiesta successiva tramite l'header **Cookie**, nel rispetto delle regole di dominio e percorso.

Lo scopo principale dei cookie è quello di **mantenere informazioni di stato** in un protocollo che, per definizione, è **stateless**.

2.6.1 Struttura

La struttura di un cookie comprende:

- **key**: nome del cookie, utilizzato per identificarlo;
- **value**: valore associato alla chiave;
- **path**: specifica il percorso del server per cui il cookie è valido;
- **domain**: definisce il dominio per cui il cookie viene inviato;
- **max-age**: indica la durata del cookie in secondi;
- **secure**: se presente, il cookie viene trasmesso solo su connessioni HTTPS;
- **version**: versione del formato del cookie (poco utilizzata nelle implementazioni moderne).

In base alla presenza o meno degli attributi di durata (**max-age** o **expires**), un cookie può essere:

- **di sessione**: eliminato alla chiusura del browser;
- **persistente**: memorizzato fino alla scadenza indicata.

2.7 Autenticazione

In molte applicazioni web è necessario implementare meccanismi di **autenticazione** per proteggere l'accesso a determinate risorse e garantire che solo utenti autorizzati possano utilizzarle.

Le tecniche di autenticazione più comuni includono:

- **Filtro degli indirizzi IP**: l'accesso viene consentito solo a richieste provenienti da specifici indirizzi IP. Questa tecnica è poco affidabile in presenza di indirizzi non pubblici, assegnati dinamicamente, NAT o tecniche di spoofing.
- **Form con username e password**: l'utente inserisce le proprie credenziali tramite un modulo HTML, che vengono poi verificate dal server. È la soluzione più diffusa e flessibile, ma richiede l'uso di HTTPS per garantire la sicurezza delle credenziali.

- **HTTP Basic Authentication:** le credenziali vengono inviate dal client al server all'interno degli header HTTP, codificate in Base64. È semplice da implementare ma insicura se non utilizzata su connessioni HTTPS.
- **HTTP Digest Authentication:** migliora la sicurezza rispetto all'autenticazione Basic utilizzando un meccanismo di hash crittografico delle credenziali, evitando l'invio diretto della password.

2.8 SSL/TLS

SSL (*Secure Socket Layer*) e TLS (*Transport Layer Security*) sono protocolli crittografici progettati per garantire la sicurezza delle comunicazioni su rete.

In particolare, SSL/TLS consentono di assicurare:

- **Confidenzialità:** i dati trasmessi sono cifrati e non leggibili da soggetti non autorizzati;
- **Integrità:** i messaggi non possono essere alterati durante il trasferimento senza che la modifica venga rilevata;
- **Autenticità:** il client può verificare l'identità del server (e opzionalmente viceversa).

Le comunicazioni protette tramite SSL/TLS avvengono utilizzando il protocollo HTTPS, ovvero HTTP incapsulato all'interno di un canale sicuro.

Il funzionamento si basa su un meccanismo di **crittografia ibrida**: la crittografia a chiave pubblica e privata viene utilizzata durante la fase iniziale di handshake per autenticare il server e scambiare in modo sicuro una chiave di sessione, mentre la comunicazione successiva utilizza crittografia simmetrica, più efficiente dal punto di vista computazionale.

L'autenticazione del server è garantita tramite un **certificato digitale**, rilasciato da una *Certification Authority* (CA) riconosciuta.

2.9 Architetture

Le architetture HTTP definiscono il ruolo degli intermediari coinvolti nella comunicazione tra client e server. Le più comuni architetture HTTP sono:

- **Proxy:** è un intermediario che riceve le richieste dal client e le inoltra al server finale. Può interpretare, filtrare, modificare o riscrivere le richieste e le risposte. È spesso utilizzato per caching, controllo degli accessi, logging e sicurezza.
- **Gateway:** funge da punto di accesso verso altri sistemi o protocolli. Dal punto di vista del client appare come un server, ma internamente può tradurre le richieste HTTP verso protocolli diversi (ad esempio database, servizi legacy o altri sistemi applicativi).
- **Tunnel:** stabilisce un canale di comunicazione trasparente tra client e server senza interpretare il contenuto dei messaggi. È tipicamente utilizzato per il trasporto di comunicazioni cifrate, come nel caso di HTTPS attraverso proxy tramite il metodo CONNECT.

2.10 Caching

Il caching in HTTP è un meccanismo volto a ridurre la latenza, il carico sul server e il consumo di banda, memorizzando copie locali o intermedie delle risorse richieste.

Le principali implementazioni del caching in HTTP sono:

- **User Agent Cache:** lo user agent (tipicamente il browser) memorizza localmente le risorse richieste in precedenza. Questo permette di velocizzare l'accesso alle pagine già visitate e ridurre il consumo di banda. È particolarmente utile su dispositivi mobili o in reti a bassa velocità.
- **Proxy Cache:** un intermediario tra client e server può memorizzare risorse richieste frequentemente. Si distinguono principalmente due tipi:
 - **Forward Proxy Cache:** intercetta le richieste provenienti dai client e memorizza le risorse più richieste. Serve a ridurre il traffico verso Internet e migliorare le prestazioni dei client interni a una rete aziendale.

- **Reverse Proxy Cache:** collocato davanti al server, riceve tutte le richieste in ingresso e può rispondere direttamente da cache senza coinvolgere il server backend. Migliora le prestazioni del server e riduce il carico di elaborazione.

2.10.1 Meccanismi collaterali

HTTP definisce inoltre meccanismi che regolano l'efficacia e la validità della cache:

- **Freshness:** indica per quanto tempo una risorsa memorizzata in cache può essere considerata valida senza dover contattare il server.
- **Validation:** consente di verificare se la risorsa memorizzata in cache è ancora valida mediante richieste condizionali al server (ad esempio tramite **ETag** o **Last-Modified**).
- **Invalidation:** permette di rimuovere o aggiornare risorse dalla cache quando diventano obsolete o vengono modificate sul server.

3 Html

4 CSS

5 Web Dinamico

6 Servlet

6.1 Web Application

Una *Web Application* è un insieme di risorse lato server che, lavorando insieme, consentono la creazione di un'applicazione web. Le risorse possono includere classi Java, pagine JSP, contenuti statici (come file HTML, CSS, JavaScript, immagini) e altri componenti necessari al funzionamento dell'applicazione.

Tali applicazioni web vengono eseguite all'interno di un *Web Container*, che si occupa di gestire il ciclo di vita delle servlet, l'invocazione delle JSP e la comunicazione tra client e server.

6.2 Servlet

Una *Servlet* è una classe Java eseguita all'interno di un *Web Container*, che fornisce un servizio di comunicazione tra client e server tramite il modello *request/response*.

Le servlet superano i limiti strutturali delle tradizionali *CGI* e consentono l'implementazione di applicazioni web dinamiche, garantendo migliori prestazioni e una gestione più efficiente delle risorse.

6.2.1 Gerarchia

Le *Servlet* possono, in generale, elaborare diversi tipi di comunicazione tra client e server. Il caso più comune è rappresentato dal protocollo HTTP, gestito tramite la classe `HttpServlet` e, più in generale, attraverso le API fornite dal package `javax.servlet.http`.

6.3 Request e Response

Alla ricezione di una richiesta da parte di un client, il *Web Container* crea un oggetto di tipo `HttpServletRequest` e lo passa alla *Servlet* designata. L'oggetto `Request` incapsula tutte le informazioni relative alla richiesta effettuata dal client, come parametri, intestazioni (*headers*), metodo HTTP utilizzato, URL richiesto e dati inviati nel corpo della richiesta.

In modo simmetrico, la *Servlet* utilizza un oggetto di tipo `HttpServletResponse` per costruire e inviare la risposta al client. L'oggetto `Response` consente di impostare il codice di stato, le intestazioni e il contenuto della risposta, come pagine HTML, dati JSON o altri tipi di risorse.

6.4 Ciclo di vita

L'intero ciclo di vita di una *Servlet* è gestito dal *Servlet Container*, il quale si occupa di crearne, inizializzarne, eseguirne e distruggerne le istanze quando necessario.

Durante la fase di inizializzazione, il container invoca il metodo `init()`, che viene eseguito una sola volta per configurare la servlet. Successivamente, per ogni richiesta proveniente da un client, vengono chiamati i metodi `service()` o, più specificamente, `doGet()` e `doPost()`, ai quali vengono passati gli oggetti `HttpServletRequest` e `HttpServletResponse`.

Infine, quando la servlet viene rimossa dal servizio, il container invoca il metodo `destroy()`, permettendo di liberare eventuali risorse allocate.

6.5 Multithreading

Intuitivamente si potrebbe pensare che, per ogni richiesta, venga creata una nuova istanza di *Servlet*. In realtà, nel modello di esecuzione standard, il *Servlet Container* utilizza una singola istanza della servlet per gestire più richieste concorrenti, assegnando a ciascuna un thread distinto.

Questo approccio consente una gestione efficiente delle risorse, ma introduce problematiche di concorrenza: i metodi `doGet()` e `doPost()` possono essere eseguiti simultaneamente da più thread sulla stessa istanza. Per questo motivo, è necessario prestare attenzione all'accesso a variabili condivise e sincronizzare eventuali sezioni critiche del codice per evitare condizioni di race e comportamenti imprevedibili.

6.5.1 Modello Single-Threaded

In alternativa al modello *multithreading*, i *Servlet Container* possono assegnare un'istanza distinta della *Servlet* a ciascun thread di richiesta. Questo approccio è noto come *Single-Threaded Model* e garantisce che ogni richiesta venga gestita da un'istanza separata, evitando problemi di concorrenza.

Tuttavia, tale modello è considerato *deprecated* poiché comporta un notevole aumento del consumo di risorse e una riduzione delle prestazioni complessive. Una *Servlet* che desidera operare in modalità *single-threaded* deve implementare l'interfaccia `SingleThreadModel`.

6.6 Metodi di controllo

La superclasse `GenericServlet` definisce i principali metodi che regolano il ciclo di vita di una *Servlet*:

- `init()`: inizializza l'istanza della servlet e viene eseguito una sola volta all'avvio;
- `service()`: metodo astratto incaricato di gestire le richieste dei client, instradandole verso l'elaborazione appropriata;
- `destroy()`: contiene le istruzioni da eseguire prima della distruzione della servlet, ad esempio per liberare risorse.

La classe `HttpServlet`, che estende `GenericServlet`, aggiunge metodi specifici per la gestione dei diversi tipi di richieste HTTP:

- `doGet()`: gestisce le richieste HTTP di tipo GET;
- `doPost()`: gestisce le richieste HTTP di tipo POST;
- `doPut()`: gestisce le richieste HTTP di tipo PUT;
- `doDelete()`: gestisce le richieste HTTP di tipo DELETE.

6.7 Response

L'oggetto `Response` viene utilizzato dalla *Servlet* per comunicare con il client. Esso implementa l'interfaccia `HttpServletResponse` e contiene le informazioni necessarie per costruire la risposta HTTP, tra cui:

- lo *status code*, che indica l'esito dell'elaborazione della richiesta;
- gli *header* del messaggio HTTP, che forniscono metadati sulla risposta;
- il *body* della risposta, che rappresenta il contenuto effettivo inviato al client.

L'interfaccia `HttpServletResponse` consente inoltre di modificare i campi dell'header, come il *Content-Type*, e di ottenere uno stream di output per scrivere il contenuto in uscita. Offre anche metodi per la gestione dei cookie e per l'invio di codici di stato o reindirizzamenti al client.

6.7.1 Status code

La gestione dello stato della risposta HTTP avviene tramite il metodo `public void setStatus(int statusCode)` dell'oggetto `HttpServletResponse`. In alternativa, è possibile utilizzare i metodi `public void sendError(int code)` e `public void sendError(int code, String message)`, che inviano direttamente un errore HTTP al client.

I codici di stato HTTP sono suddivisi in cinque categorie principali:

- **1xx – Informational**: indicano informazioni sullo stato della richiesta in corso;
- **2xx – Success**: indicano che la richiesta è stata elaborata con successo (ad esempio, 200 OK);
- **3xx – Redirection**: indicano che è necessaria un'azione aggiuntiva per completare la richiesta, come un reindirizzamento;
- **4xx – Client Error**: indicano errori dovuti al client, come richieste malformate o risorse non trovate (ad esempio, 404 Not Found);
- **5xx – Server Error**: indicano errori lato server durante l'elaborazione della richiesta (ad esempio, 500 Internal Server Error).

6.7.2 Header

Per la gestione degli header HTTP, l'oggetto `HttpServletResponse` mette a disposizione diversi metodi, tra cui:

- `setHeader(String name, String value)`: imposta un header con il nome e il valore specificati, sovrascrivendo eventuali valori precedenti;
- `setDateHeader(String name, long date)`: imposta un header di tipo data;
- `setIntHeader(String name, int value)`: imposta un header di tipo intero;
- `addHeader(String name, String value), addDateHeader(String name, long date), addIntHeader(String name, int value)`: aggiungono un valore all'header specificato senza sovrascrivere i precedenti;
- `setContentType(String type)`: imposta il tipo MIME della risposta, ad esempio `"text/html"` o `"application/json"`;
- `setContentLength(int length)`: imposta la lunghezza del contenuto della risposta;
- `addCookie(Cookie cookie)`: aggiunge un cookie alla risposta;
- `sendRedirect(String location)`: invia un reindirizzamento HTTP al client verso l'URL specificato.

6.7.3 Contenuto

Per aggiungere contenuto alla risposta HTTP, l'oggetto `HttpServletResponse` mette a disposizione due tipi di stream:

- `public PrintWriter getWriter()`: restituisce un `PrintWriter` per scrivere dati di testo (ad esempio HTML o JSON) nella risposta;
- `public ServletOutputStream getOutputStream()`: restituisce un `ServletOutputStream` per scrivere dati binari (ad esempio immagini o file) nella risposta.

È importante non utilizzare entrambi gli stream contemporaneamente sulla stessa risposta, poiché ciò può generare eccezioni.

6.8 Request

L'oggetto `Request`, analogamente a `Response`, viene utilizzato per la comunicazione tra client e server, questa volta nel senso dal client verso il server. Nelle comunicazioni HTTP, esso è rappresentato dall'interfaccia `HttpServletRequest`, che fornisce numerosi metodi per accedere e manipolare le informazioni della richiesta. Tra i principali elementi contenuti in un oggetto `HttpServletRequest` troviamo:

- **URL**: l'indirizzo richiesto dal client;
- **Header**: informazioni aggiuntive inviate dal client, come tipo di contenuto, agent e cache;
- **Auth type e informazioni sull'utente**: tipo di autenticazione e dati dell'utente autenticato;
- **Cookie**: eventuali cookie inviati dal client;
- **Session**: riferimento alla sessione associata alla richiesta, se presente.

6.8.1 URL

La struttura di un URL per una richiesta HTTP è la seguente:

`http://[host]:[port]/[path]?[query]`

In particolare:

- **[path]**: indica il contesto dell'applicazione web e il nome della risorsa richiesta (ad esempio, una servlet o una pagina);
- **[query]**: contiene i parametri inviati dal client, sotto forma di coppie chiave-valore separate da `&`.

I principali metodi forniti dall'oggetto `HttpServletRequest` per accedere alle informazioni dell'URL sono:

- `public String getParameter(String name)`: restituisce il valore di un parametro della query string;

- `public String getContextPath():` restituisce il contesto dell'applicazione web;
- `public String getQueryString():` restituisce la query string completa dell'URL;
- `public String getPathInfo():` restituisce la parte di path relativa alla servlet;
- `public String getPathTranslated():` restituisce il path tradotto in percorso assoluto sul file system, se disponibile.

6.8.2 Autenticazione, sicurezza e cookie

Oltre ai metodi per gestire l'URL e i parametri, l'oggetto `HttpServletRequest` mette a disposizione metodi per gestire autenticazione, sicurezza e cookie:

- `String getRemoteUser():` restituisce il nome dell'utente autenticato, se presente;
- `String getAuthType():` restituisce il tipo di autenticazione utilizzata (ad esempio BASIC, FORM, DIGEST, CLIENT-CERT);
- `boolean isUserInRole(String role):` verifica se l'utente autenticato appartiene a un determinato ruolo;
- `Cookie[] getCookies():` restituisce un array di cookie inviati dal client nella richiesta.

6.8.3 Contenuto

Particolarmente rilevante per le richieste di tipo *POST*, l'interfaccia `HttpServletRequest` permette di accedere al contenuto (body) della richiesta HTTP tramite il metodo:

- `ServletInputStream getInputStream():` restituisce uno stream di input per leggere i dati binari inviati dal client, come form multipart, file o altri contenuti non testuali.

Per i dati testuali, come i form codificati in `application/x-www-form-urlencoded`, è possibile utilizzare anche il metodo `getReader()`, che restituisce un `BufferedReader`.

6.9 Deployment

Il termine *deployment* indica il processo di installazione di una *Servlet* all'interno di un *Servlet Container*. Tale processo comprende:

- la definizione del *runtime environment* necessario per l'esecuzione della servlet;
- la mappatura delle URL, che associa le richieste HTTP alle servlet corrispondenti;
- la definizione delle impostazioni di default per la servlet e l'applicazione web;
- la configurazione delle caratteristiche di sicurezza, come ruoli, autenticazione e accesso alle risorse.

6.9.1 Web Archives

I *Web Archives* (file con estensione `.war`) definiscono la distribuzione e il deployment delle applicazioni web. Si tratta di file *JAR* che seguono una struttura ben definita:

- **META-INF:** contiene informazioni di configurazione generali sul pacchetto;
- **WEB-INF:** directory principale per le risorse server-side, al cui interno troviamo:
 - **classes:** contiene le classi compilate delle servlet e altri file `.class`;
 - **lib:** contiene le librerie esterne necessarie all'applicazione;
 - **web.xml:** file di deployment descriptor che permette di configurare l'applicazione web.

Il file `web.xml` contiene l'elenco delle servlet e, per ciascuna, definisce:

- il nome della servlet;
- la classe Java corrispondente;
- eventuali parametri di configurazione;
- la mappatura delle URL a cui la servlet risponde.

6.9.2 Configurazione della Servlet

Per ogni *Servlet* è possibile definire dei parametri di configurazione. Questi vengono specificati nel file `web.xml` e seguono la seguente struttura:

```
<init-param>
  <param-name>parName</param-name>
  <param-value>parValue</param-value>
</init-param>
```

All'interno della servlet, è poi possibile accedere a tali parametri tramite l'interfaccia `ServletConfig`. Questa viene passata come parametro al metodo `init()` e rimane sempre accessibile attraverso il metodo `getServletConfig()`.

In particolare, per ottenere il valore di un parametro di inizializzazione si utilizza il metodo:

- **String** `getInitParameter(String name)`: restituisce il valore del parametro identificato da `name`.

6.9.3 Contesto

Per ogni applicazione web è possibile definire un contesto tramite il file `web.xml`. Il contesto può contenere parametri di inizializzazione e attributi. I parametri di inizializzazione a livello di contesto possono essere definiti come segue:

```
<web-app>
  <context-param>
    <param-name>feedback</param-name>
    <param-value>feedback@deis.unibo.it</param-value>
  </context-param>
  ...
</web-app>
```

Gli attributi di contesto rappresentano, invece, dati globali che possono essere letti e modificati da tutte le servlet dell'applicazione.

È possibile accedere al contesto tramite l'interfaccia `ServletContext`, ottenuta con il metodo `getServletContext()` di una servlet. `ServletContext` permette anche di accedere alle risorse dell'applicazione come stream tramite il metodo:

- **InputStream** `getResourceAsStream(String path)`: restituisce uno stream per leggere il contenuto di una risorsa situata nel percorso specificato all'interno del contesto web.

Gli oggetti `ServletContext` sono condivisi tra tutte le servlet e tutti gli utenti dell'applicazione, consentendo la gestione di dati globali.

6.10 Sessione

A differenza delle semplici richieste HTTP, che sono *stateless*, molte applicazioni web richiedono la possibilità di mantenere informazioni tra più richieste dello stesso client. A questo scopo si utilizza il concetto di *sessione* (*session tracking*).

La gestione della sessione può essere implementata principalmente attraverso due tecniche:

- **Cookie**: un identificatore univoco della sessione viene memorizzato nel browser del client e inviato al server ad ogni richiesta successiva;
- **URL rewriting**: l'identificatore della sessione viene aggiunto direttamente all'URL delle richieste, consentendo di tracciare la sessione anche quando i cookie non sono disponibili.

6.10.1 Cookie

La classe `Cookie` in Java rappresenta un cookie HTTP, ossia una coppia **nome-valore** a cui possono essere associati attributi come durata, path, dominio e flag di sicurezza.

Per gestire i cookie nelle servlet, gli oggetti `HttpServletRequest` e `HttpServletResponse` mettono a disposizione i seguenti metodi:

- `Cookie[] getCookies()`: restituisce tutti i cookie inviati dal client nella richiesta corrente;
- `void addCookie(Cookie cookie)`: aggiunge un cookie alla risposta, che verrà inviato al client.

6.10.2 Web Container

Le sessioni nelle applicazioni web sono gestite dal *Web Container* e possono essere condivise tra tutte le servlet appartenenti alla stessa applicazione. Una sessione può contenere dati di varia natura, come attributi specifici dell'utente, preferenze o informazioni di stato. Ogni sessione è sempre identificata in modo univoco tramite un *ID di sessione*, che permette al container di associare le richieste successive dello stesso client alla stessa sessione.

6.10.3 Accesso

Per accedere alle sessioni, le servlet utilizzano l'oggetto `HttpSession`, ottenibile tramite il metodo:

- `HttpSession getSession(boolean createNew)` di `HttpServletRequest`.

Il parametro booleano `createNew` indica se, in assenza di una sessione esistente, il metodo deve crearne una nuova (`true`) oppure restituire `null` (`false`).

6.10.4 Metodi

L'oggetto `HttpSession` mette a disposizione diversi metodi utili per gestire le sessioni, tra cui:

- `String getId()`: restituisce l'ID univoco della sessione;
- `boolean isNew()`: indica se la sessione è stata appena creata;
- `void invalidate()`: invalida la sessione corrente, rimuovendo tutti gli attributi associati;
- `long getCreationTime()`: restituisce il timestamp di creazione della sessione;
- `long getLastAccessedTime()`: restituisce il timestamp dell'ultima richiesta ricevuta relativa alla sessione.

6.11 Scope differenziati

Le classi `ServletContext`, `HttpSession` e `HttpServletRequest` offrono tutte metodi per memorizzare e accedere ai dati, ma ciascuna gestisce uno scope differente. In particolare, lo *scope* di un dato è definito dal suo tempo di vita e dall'accessibilità all'interno dell'applicazione:

- **ServletContext**: scope globale all'applicazione, condiviso tra tutte le servlet e tra tutti gli utenti, con durata pari a quella dell'applicazione;
- **HttpSession**: scope legato a una specifica sessione utente, condiviso tra tutte le servlet dell'applicazione per quello stesso utente, con durata pari a quella della sessione;
- **HttpServletRequest**: scope limitato alla singola richiesta, valido solo durante l'elaborazione della request.

6.11.1 Funzionalità degli oggetti scoped

Tutti gli oggetti *scoped* offrono metodi comuni per gestire i dati associati al loro ambito:

- `void setAttribute(String name, Object o)`: associa un oggetto all'identificativo **name** all'interno dello scope;

- `Object getAttribute(String name)`: recupera l'oggetto associato al nome specificato;
- `void removeAttribute(String name)`: rimuove l'oggetto associato al nome specificato;
- `Enumeration getAttributeNames()`: restituisce un elenco dei nomi di tutti gli attributi presenti nello scope.

6.12 Inclusione di risorse

All'interno di una pagina web è possibile includere sia risorse statiche che dinamiche. Le risorse statiche possono essere rappresentate da semplici file HTML, immagini o altri contenuti non elaborati dal server. Le risorse dinamiche, invece, possono essere generate da altre servlet o da pagine JSP.

Per includere una risorsa all'interno della risposta corrente si utilizza l'interfaccia `RequestDispatcher`, invocandone il metodo:

- `void include(ServletRequest request, ServletResponse response)`

In questo modo, l'output della risorsa inclusa viene inserito nel flusso di risposta generato dalla servlet chiamante, senza interromperne l'esecuzione.

6.13 Inoltro

L'inoltro (*forward*) viene utilizzato quando una servlet gestisce solo parzialmente una richiesta e desidera demandarne la continuazione o la conclusione a un'altra risorsa (ad esempio un'altra servlet o una JSP). Anche in questo caso si utilizza l'interfaccia `RequestDispatcher`, ma attraverso il metodo:

- `void forward(ServletRequest request, ServletResponse response)`

L'inoltro interrompe l'esecuzione della servlet chiamante e trasferisce completamente il controllo alla risorsa destinazione, la quale genera la risposta finale per il client.

6.14 Redirezione

La *redirezione* (*redirect*) è un meccanismo che consente al server di istruire il client a effettuare una nuova richiesta verso una risorsa diversa. A differenza del `forward`, che avviene interamente sul lato server, la redirezione implica un nuovo ciclo di richiesta e risposta HTTP, con conseguente aggiornamento dell'URL nel browser.

In Java, la redirezione viene effettuata tramite l'oggetto `HttpServletResponse`, invocando il metodo:

- `void sendRedirect(String location)`

Quando questo metodo viene chiamato, il server invia al client un codice di stato HTTP 302 (`Found`) o 303 (`See Other`), insieme all'intestazione `Location` contenente il nuovo URL di destinazione. Il browser, ricevuta la risposta, effettua una nuova richiesta verso la risorsa indicata.

7 Java Server Pages

Le *Java Server Pages* (JSP) permettono di estendere l'HTML con codice Java, consentendo la generazione dinamica di contenuti. In una JSP, il codice HTML viene inviato direttamente nello stream di output, mentre il codice Java viene prima eseguito sul server e il suo risultato viene incorporato nella risposta.

Durante il deployment, ogni pagina JSP viene compilata dal container in una servlet equivalente: il markup HTML diventa output statico, mentre le porzioni di codice Java vengono tradotte.

7.1 JSP Servlet

Le richieste verso una pagina JSP vengono gestite da una servlet speciale generata automaticamente dal *Web Container*. In generale, il container esegue le seguenti operazioni:

- **Traduzione** della pagina JSP in una servlet Java equivalente;
- **Compilazione** della servlet generata;
- **Esecuzione** della servlet, che produce la risposta HTML finale.

Questo processo avviene automaticamente alla prima richiesta della JSP (o quando la pagina viene modificata). Le richieste successive utilizzano direttamente la servlet già compilata.

7.2 Differenza con le Servlet

La differenza principale tra JSP e Servlet riguarda il modo in cui viene implementata la logica applicativa e generata la risposta.

Nelle **Servlet**, l'intera risposta HTML deve essere prodotta tramite codice Java: questo comporta la creazione esplicita di stringhe HTML all'interno del codice, un processo ripetitivo, poco leggibile, soggetto ad errori e difficilmente scalabile, soprattutto in applicazioni complesse.

Le **JSP**, invece, permettono di mantenere la struttura della pagina direttamente in HTML, inserendo codice Java solo per le parti dinamiche. In questo modo la separazione dei compiti è più chiara: gli sviluppatori possono concentrarsi sulla logica, mentre i designer possono lavorare direttamente sul markup, migliorando la manutenibilità e la produttività complessiva del progetto.

7.2.1 Flusso

Nelle JSP l'ordine di esecuzione delle operazioni è fondamentale. Poiché l'output della JSP viene inviato al client man mano che viene generato, l'header della risposta HTTP deve essere inviato per primo, prima di qualsiasi contenuto HTML. Per questo motivo, una volta che la generazione dell'output è iniziata, non è più possibile modificare gli header della risposta né effettuare un inoltro o includere un'altra JSP che richieda un nuovo flusso di output.

In altre parole, dopo l'inizio della produzione del contenuto, la servlet generata dalla JSP può solo continuare a scrivere nel flusso di risposta già aperto.

7.3 Tag

Le sezioni dinamiche all'interno delle pagine JSP possono essere definite secondo due diversi insiemi di tag:

- **Scripting-oriented**: derivati direttamente dalla sintassi delle servlet, permettono l'inserimento di codice Java all'interno della pagina;
- **XML-oriented**: rappresentano la versione XML dei tag JSP, più strutturata e compatibile con strumenti che richiedono documenti XML ben formati.

7.3.1 Scripting-oriented

I tag scripting-oriented permettono l'inserimento esplicito di codice Java all'interno della JSP. Essi includono:

- **<%! %> — Dichiarazione**: consente di dichiarare variabili e metodi a livello di classe della servlet generata;

- `<%= %>` — **Espressione**: valuta l'espressione Java e ne inserisce il risultato nell'output;
- `<% %>` — **Scriptlet**: contiene codice Java eseguito nel metodo `service()` della servlet generata;
- `<%@ %>` — **Direttiva**: imposta informazioni di configurazione della JSP (es. `page`, `include`, `taglib`).

7.3.2 XML-oriented

I tag XML-oriented rappresentano l'equivalente XML dei tag JSP scripting-oriented e permettono di scrivere JSP come documenti XML validi:

- `<jsp:declaration>...</jsp:declaration>` — dichiarazione a livello di classe;
- `<jsp:expression>...</jsp:expression>` — espressione valutata e inserita nell'output;
- `<jsp:scriptlet>...</jsp:scriptlet>` — blocco di codice Java eseguito nel corpo della servlet;
- `<jsp:directive.dir.type ... />` — direttiva JSP in forma XML, dove `dir.type` può essere `page`, `include` o `taglib`.

7.4 Direttive

Le direttive JSP vengono definite tramite il tag `<%@ ... %>` e consentono di configurare il comportamento della pagina JSP o del processo di traduzione/compilazione. Le direttive principali sono:

- `page`: definisce attributi e proprietà valide per l'intera pagina;
- `include`: include staticamente un altro file durante la fase di traduzione;
- `taglib`: rende disponibile una libreria di tag personalizzati.

7.4.1 Page

La direttiva `page` definisce attributi che influiscono sull'intera JSP. Gli attributi principali sono:

- `language="java"`: specifica il linguaggio utilizzato nel codice della JSP (normalmente sempre `java`);
- `import="package.class | package.*"`: elenca le classi o i package da importare, in modo analogo all'istruzione `import` di Java; è possibile specificare più import separati da virgole;
- `session="true | false"`: indica se la pagina utilizza l'oggetto implicito `session`. Se impostato a `false`, l'oggetto `session` non viene creato;
- `isThreadSafe="true | false"`: se `false`, la servlet generata implementa `SingleThreadModel` (deprecato), garantendo che solo un thread per volta la esegua;
- `errorPage="url"`: specifica la pagina che deve essere caricata in caso di eccezioni non gestite;
- `isErrorPage="true | false"`: indica se la pagina è una pagina d'errore e consente di utilizzare l'oggetto implicito `exception`;
- `contentType="mime/type; charset=..."`: definisce il tipo MIME della risposta e il set di caratteri;
- `pageEncoding="charset"`: specifica l'encoding del file JSP stesso;
- `buffer="size"`: dimensione del buffer di output (ad esempio `"8kb"`); l'output non può essere inviato prima del riempimento del buffer;
- `autoFlush="true | false"`: specifica se il buffer deve essere svuotato automaticamente quando pieno;
- `extends="className"`: permette di definire una superclasse personalizzata per la servlet generata;
- `info="text"`: definisce una stringa di descrizione accessibile tramite `getServletInfo()`.

7.4.2 Include

Attraverso la direttiva `include` è possibile includere staticamente all'interno della JSP il contenuto di altri file (tipicamente frammenti HTML, JSP o porzioni comuni). L'inclusione avviene **in fase di traduzione**, prima della compilazione, e quindi il contenuto del file incluso viene incorporato direttamente nel codice sorgente della servlet generata.

- Sintassi: `<%@ include file="percorso" %>`
- L'inclusione è **statica**: il file incluso è copiato nel sorgente della servlet.
- È possibile nidificare un numero arbitrario di inclusioni.
- Una modifica a un file incluso richiede la **ricompilazione di tutte le JSP** che lo includono.

Questo meccanismo è tipicamente usato per includere header, footer e blocchi ricorrenti dell'interfaccia grafica.

7.4.3 Taglib

La direttiva `taglib` permette di includere una libreria di tag personalizzati (*custom tag*) che estendono la sintassi JSP con nuove funzionalità. Le librerie di tag sono definite tramite:

- una classe Java che implementa la logica del tag;
- un file *TLD* (Tag Library Descriptor) che descrive i tag disponibili.

Sintassi:

- `<%@ taglib uri="uri-della-libreria" prefix="pref" %>`

Dopo l'inclusione, i tag definiti nella libreria possono essere utilizzati nella JSP tramite il prefisso specificato.

7.5 Built-in objects

JSP mette a disposizione una serie di *built-in objects* (o *implicit objects*), già istanziati dal container e immediatamente utilizzabili all'interno della pagina, senza necessità di dichiarazioni esplicite. Essi consentono di accedere ai principali elementi del modello servlet.

- **request**: oggetto `HttpServletRequest` relativo alla richiesta corrente.
- **response**: oggetto `HttpServletResponse` associato alla risposta HTTP.
- **session**: oggetto `HttpSession` associato all'utente (se `session="true"` nella direttiva `page`).
- **application**: oggetto `ServletContext` dell'intera applicazione web, utile per ottenere informazioni di contesto.
- **out**: istanza di `JspWriter` utilizzata per generare l'output verso il client.
- **config**: oggetto `ServletConfig` associato alla JSP, contenente i parametri di configurazione della servlet generata.
- **page**: riferimento all'istanza della servlet generata dalla JSP (equivalente a `this`).
- **pageContext**: oggetto `PageContext` che fornisce accesso ai diversi scope (`page`, `request`, `session`, `application`). Incapsula tutti gli implicit objects e permette anche il trasferimento del controllo ad altre pagine.
- **exception**: disponibile solo nelle pagine di errore (direttiva `isErrorPage="true"`); rappresenta l'eccezione non gestita che ha causato il trasferimento alla pagina d'errore.

7.6 Azioni

Le *azioni JSP* (JSP Actions) sono costrutti che consentono di eseguire operazioni dinamiche all'interno di una pagina JSP. Esse permettono la comunicazione tra componenti, la gestione di JavaBean, e l'interazione con altre risorse del server. Le azioni sono sempre racchiuse tra tag XML e vengono elaborate dal container JSP durante la fase di esecuzione.

In generale esistono sei tipi principali di azioni:

- **useBean**: crea o recupera un JavaBean e lo rende disponibile alla pagina JSP.
- **getProperty**: legge una proprietà del JavaBean e ne restituisce il valore.
- **setProperty**: imposta una proprietà del JavaBean, eventualmente leggendo i valori dai parametri della richiesta.
- **include**: include dinamicamente il contenuto di un'altra risorsa (JSP, HTML o servlet) durante l'esecuzione.
- **forward**: inoltra la richiesta a un'altra risorsa del server, interrompendo l'esecuzione della pagina corrente.
- **plugin**: genera in automatico il codice HTML necessario per caricare applet Java o componenti basati su plugin.

7.6.1 Forward

L'azione **forward** consente di trasferire il controllo dell'esecuzione ad un'altra risorsa sul server, a condizione che non sia stato ancora prodotto alcun output verso il client. L'attributo *page* specifica l'URL relativo della risorsa di destinazione.

Quando avviene il **forward**, la *request* e la *response* non vengono restituite al client: il trasferimento avviene completamente sul lato server. La risorsa di destinazione riceve la stessa **request** e può accedere ai suoi parametri. Viene invece istanziato un nuovo **pageContext**.

La sintassi è la seguente:

```
<jsp:forward page="url" />
```

È possibile generare dinamicamente il valore dell'attributo *page* e aggiungere parametri al trasferimento tramite l'azione **jsp:param**, come nel seguente esempio:

```
<jsp:forward page="url">
  <jsp:param name="name" value="value" />
</jsp:forward>
```

7.6.2 Include

L'azione **include** permette di includere dinamicamente nella pagina corrente il contenuto generato da un'altra risorsa locale. Il controllo dell'esecuzione viene temporaneamente trasferito alla risorsa indicata nell'attributo *page*, dopodiché ritorna alla pagina chiamante.

L'attributo *flush* specifica se il buffer di output della pagina corrente debba essere svuotato prima dell'inclusione. Se impostato a **true**, tutto l'output generato fino a quel punto viene inviato al client prima dell'inclusione della risorsa.

La sintassi è la seguente:

```
<jsp:include page="url" flush="true" />
```

È inoltre possibile passare parametri alla risorsa inclusa tramite **jsp:param**, come nel seguente esempio:

```
<jsp:include page="url" flush="true">
  <jsp:param name="name" value="value" />
</jsp:include>
```

Così come per **forward**, anche **include** condivide gli oggetti **session** e **request** con la risorsa inclusa, ma crea un nuovo **pageContext**.

7.7 Modello a componenti

JSP consente uno sviluppo centrato sulla pagina, basato su un modello a componenti che prevede una netta separazione tra logica applicativa e presentazione. In questo approccio, la pagina JSP si occupa principalmente della generazione dell'interfaccia utente, mentre la logica di business viene delegata a componenti esterni, come servlet o JavaBean.

Questa separazione favorisce la modularità, il riuso del codice e la manutenibilità, permettendo la realizzazione di architetture web più articolate e scalabili.

7.7.1 Proprietà

Le proprietà rappresentano elementi dello stato che vengono esposti in modo controllato. In Java, in particolare, esse non costituiscono una costruzione del linguaggio ma seguono una convenzione basata sui metodi di accesso. In altri linguaggi, come C#, le proprietà sono invece definite in modo rigoroso dal linguaggio stesso.

Una proprietà `prop` è generalmente associata ai metodi `getProp()` e `setProp()`. Se viene definito soltanto il metodo `get`, la proprietà è di tipo *read-only*. Le proprietà possono inoltre essere indicizzate, assumendo un comportamento simile a quello di un pseudo-array (`getProp(int index)`).

7.7.2 Componenti

I componenti sono strutture che operano all'interno di container, i quali ne gestiscono il ciclo di vita e i collegamenti con altri componenti. Poiché i container non conoscono in anticipo i tipi dei componenti con cui dovranno interagire, la comunicazione avviene in maniera dinamica tramite meccanismi di *reflection*.

Un contenitore di bean, detto *bean container*, è in grado di interfacciarsi con i componenti mediante la *Java Reflection*, che mette a disposizione strumenti di introspezione e dispatching. Questo permette al container di analizzare le proprietà esposte dal componente e di invocare metodi in modo dinamico, senza dipendere da un tipo specifico.

7.8 JSP e Bean

JSP permette di agganciare e utilizzare i bean tramite tag specifici, che si dividono in tre categorie principali:

- Creare o recuperare un riferimento a un bean
- Impostare il valore delle proprietà di un bean
- Leggere il valore delle proprietà di un bean

7.8.1 useBean

L'azione `<jsp:useBean>` permette di creare o inizializzare un riferimento a un JavaBean all'interno di una pagina JSP. Se il bean esiste già nello *scope* specificato, viene riutilizzato; altrimenti viene istanziato un nuovo oggetto.

I principali attributi sono:

- **id**: il nome con cui il bean sarà referenziato nella pagina JSP.
- **class**: il nome completo della classe Java del bean.
- **scope**: definisce l'ambito di visibilità del bean (`page`, `request`, `session`, `application`).

La sintassi generale è la seguente:

```
<jsp:useBean id="beanName" class="com.example.ClassName"
             scope="page | request | session | application" />
```

L'attributo `scope` consente di estendere la durata del bean:

- **page**: il bean è disponibile solo durante l'elaborazione della pagina corrente.
- **request**: il bean è disponibile per tutta la durata della richiesta HTTP corrente.
- **session**: il bean è disponibile per tutta la sessione dell'utente.
- **application**: il bean è disponibile per tutta l'applicazione web.

7.8.2 getProperty

Il tag `<jsp:getProperty>` permette di leggere il valore di una proprietà di un bean. Il valore viene direttamente scritto nell'output della pagina JSP. Questo tag non può avere un body.

I principali attributi sono:

- **name:** il nome del bean da cui leggere la proprietà.
- **property:** il nome della proprietà da leggere.

La sintassi è la seguente:

```
<jsp:getProperty name="beanName" property="propertyName" />
```

7.8.3 setProperty

Il tag `<jsp:setProperty>` permette di impostare il valore di una proprietà di un bean. Può leggere il valore da una stringa letterale o direttamente da un parametro della richiesta HTTP.

Gli attributi principali sono:

- **name:** il nome del bean su cui impostare la proprietà.
- **property:** il nome della proprietà da impostare.
- **value:** il valore da assegnare alla proprietà.
- **param:** (opzionale) il nome di un parametro della request da utilizzare come valore.

Esempi di sintassi:

```
<jsp:setProperty name="beanName" property="propertyName" value="valore" />
<jsp:setProperty name="beanName" property="propertyName" param="paramName" />
```

7.9 Model 1

La struttura J2EE a due livelli costituita da JSP e JavaBean prende il nome di *Model 1*. In questo modello, le pagine JSP gestiscono sia la presentazione sia parte della logica di controllo, mentre i JavaBean si occupano di incapsulare la logica di business e lo stato dei dati.

Questo approccio è semplice da implementare ma può risultare meno modulare e difficile da mantenere in applicazioni complesse, poiché la logica di controllo è spesso mescolata alla presentazione.

7.10 Custom tag

JSP permette la creazione di una collezione di tag personalizzati (*custom tag*) mediante classi Java. Per poterli utilizzare all'interno di una pagina JSP, è necessario dichiarare la libreria dei tag con il direttivo `taglib`:

```
<%@ taglib uri="tagUri" prefix="tagPre" %>
```

I principali attributi sono:

- **uri:** fa riferimento al file XML (TLD, Tag Library Descriptor) che contiene le informazioni sulla classe del tag.
- **prefix:** indica il prefisso da utilizzare nei tag personalizzati all'interno della pagina JSP.

Dopo questa dichiarazione, è possibile utilizzare i tag della libreria preceduti dal prefisso specificato, ad esempio:

```
<tagPre:customTag attribute="value" />
```

7.10.1 Tag library

Per definire una tag library si utilizza un file XML chiamato **TLD (Tag Library Descriptor)**, che specifica i tag, i loro attributi, il tipo di body consentito e la classe Java che gestisce ciascun tag. In particolare, un TLD contiene:

- **Tag Lib Definition:** definisce i singoli tag della libreria e il riferimento alle classi Java che li implementano.
- **Classi che gestiscono i tag:** ogni tag è associato a una classe Java che ne realizza il comportamento.

7.10.2 Implementazione tag

Per implementare tag personalizzati di base in JSP, è necessario creare una classe Java che estenda **TagSupport**. La classe deve tipicamente sovrascrivere i seguenti metodi:

- **doStartTag():** eseguito all'inizio del tag. Può utilizzare l'oggetto **out** per scrivere contenuto nella pagina JSP; se non è presente body da elaborare, normalmente ritorna **SKIP_BODY**.
- **doEndTag():** eseguito alla fine del tag; normalmente ritorna **EVAL_PAGE** per continuare l'elaborazione della pagina.

Per ogni attributo del tag, la classe deve definire i metodi **setAttrName()** e, se necessario, **getAttrName()**, in modo che il container JSP possa impostare e leggere i valori forniti nella pagina.

In questo modo, il tag può interagire con la pagina JSP e generare contenuto dinamico in maniera modulare.

8 Javascript

JavaScript è un linguaggio di scripting eseguito lato client, utilizzato per rendere le pagine web dinamiche e interattive. Il suo nome standardizzato è ECMAScript, ovvero la specifica ufficiale su cui il linguaggio è basato.

Può essere incluso all'interno delle pagine HTML oppure caricato da file esterni, ed è interpretato direttamente dal browser.

8.1 Standardizzazione

Il processo di standardizzazione di JavaScript è stato guidato da ECMA International, che ne ha definito la specifica ufficiale chiamata *ECMAScript* (ECMA-262). Le tappe principali sono:

- **1997 – ECMA-262 Edizione 1:** JavaScript diventa uno standard ufficiale.
- **1999 – ECMAScript Edizione 3:** prima versione ampiamente adottata; introduce le basi del linguaggio moderno.
- **2009 – ECMAScript Edizione 5 (ES5):** migliora la sintassi, introduce JSON, `strict mode` e molte API utili.
- **2011 – ECMAScript Edizione 5.1:** versione utilizzata come riferimento per lo standard ISO.
- **2015 – ECMAScript Edizione 6 (ES6 / ES2015):** la più importante evoluzione del linguaggio; introduce classi, moduli, `let/const`, arrow functions, Promise e molto altro.
- **Edizioni successive (ES2016+):** pubblicazione annuale con nuove funzionalità incrementali.
- **ECMAScript 2021+ (ES12 e oltre):** include ulteriori miglioramenti come gli operatori logici avanzati, Promise.any, nuovi metodi su stringhe e array, ecc.

8.2 Java

Nonostante la somiglianza nel nome, JavaScript e Java sono due linguaggi completamente differenti.

JavaScript è un linguaggio interpretato ed eseguito generalmente dal browser. È dinamicamente tipizzato (il tipo di una variabile può cambiare nel tempo) ed è considerato object-based: utilizza oggetti, ma non si basa originariamente su un modello a classi tradizionale, bensì su un meccanismo di ereditarietà prototipale. Java, al contrario, è un linguaggio compilato ed eseguito tramite la Java Virtual Machine (JVM). È staticamente tipizzato (il tipo delle variabili viene definito a tempo di compilazione) ed è completamente object-oriented, con un modello basato su classi e una tipizzazione forte e rigorosa.

8.3 Utilizzi

Nato per rendere le pagine web dinamiche e interattive, JavaScript viene eseguito dal browser e consente di svolgere numerose operazioni, tra cui:

- accedere e modificare gli elementi HTML tramite il DOM;
- reagire agli eventi generati dall'utente (click, input, movimenti del mouse, ecc.);
- validare i dati inseriti nei form prima dell'invio al server;
- interagire con il browser (gestione dei cookie, navigazione, cronologia, timer);
- effettuare comunicazioni asincrone con il server (AJAX, Fetch API);
- manipolare contenuti multimediali e grafiche (Canvas, WebGL).

8.4 Sintassi

La sintassi di JavaScript è di tipo *C-like*, quindi simile a quella di linguaggi come C, Java e C++. JavaScript è **case-sensitive** e utilizza il carattere `;` come terminatore di istruzione, sebbene nella maggior parte dei casi esso possa essere omesso grazie al meccanismo di *automatic semicolon insertion* (ASI).

I commenti sono gestiti come nei linguaggi della famiglia C:

- `//` per i commenti su una singola linea;
- `/* ... */` per i commenti su più linee.

8.4.1 Variabili

Le variabili possono essere dichiarate tramite la keyword storica **var**, seguita dal nome della variabile. Il nome può contenere lettere, cifre, il carattere `_` e il simbolo `$`, ma non può iniziare con una cifra e non può coincidere con una parola chiave riservata del linguaggio.

JavaScript è dinamicamente tipizzato: una variabile non ha un tipo statico e può contenere dati di qualunque tipo (numeri, stringhe, oggetti, funzioni, ecc.). Le variabili possono essere inizializzate contestualmente alla dichiarazione oppure assegnate successivamente.

Le variabili dichiarate con **var** seguono due tipi di scope:

- **scope globale**, se dichiarate fuori da qualsiasi funzione;
- **scope di funzione**, se dichiarate all'interno di una funzione.

Va notato che **var** non rispetta lo scope di blocco (ad esempio, all'interno di `{...}`). Le versioni moderne del linguaggio introducono le keyword **let** e **const**, che offrono uno scope più prevedibile (scope di blocco) e dovrebbero essere preferite a **var**.

8.4.2 Valori speciali

Una variabile JavaScript può assumere alcuni valori speciali:

- **null**: rappresenta l'assenza intenzionale di un dato.
- **undefined**: indica che una variabile è stata dichiarata ma non inizializzata, oppure che una proprietà o un valore atteso non esiste.

8.4.3 Tipi primitivi

JavaScript prevede un numero relativamente ridotto di tipi primitivi:

- **Number**: tutti i valori numerici come *floating point* a 64 bit (IEEE 754). Valori speciali: **NaN**, **+Infinity**, **-Infinity**.
- **Boolean**: valori logici **true** e **false**.
- **String**: sequenze di caratteri Unicode racchiuse tra doppi ("`...`") o singoli apici ('`...`').
- **Undefined**: indica una variabile dichiarata ma non inizializzata.
- **Null**: rappresenta intenzionalmente l'assenza di valore.
- **Symbol** (ES6): identificatore univoco e immutabile.

Oltre ai tipi primitivi, JavaScript supporta i tipi *object*, che comprendono array, funzioni, oggetti generici e strutture più complesse.

8.4.4 Literal

Per ogni tipo di dato esiste una forma per rappresentare i valori costanti, detta **literal**. Alcuni esempi:

- **Number**: 42, 3.14, 0xFF, NaN, Infinity
- **Boolean**: true, false
- **String**: "ciao", 'hello'
- **Null**: null
- **Undefined**: non esiste un literal esplicito
- **Symbol**: Symbol("descrizione")

8.5 Oggetti

Gli oggetti in JavaScript sono tipi composti che rappresentano insiemi dinamici di coppie *chiave-valore* (proprietà). A differenza dei linguaggi basati su classi tradizionali, gli oggetti JavaScript possono essere estesi e modificati anche dopo la loro creazione.

Un oggetto può essere creato utilizzando il costruttore `Object()`:

```
let obj = new Object();
```

Le proprietà possono essere aggiunte, modificate o rimosse dinamicamente in qualsiasi momento. L'accesso alle proprietà può avvenire in due modi:

- **dot notation:** `oggetto.proprieta`
- **bracket notation:** `oggetto["proprieta"]`, utile quando il nome della proprietà contiene spazi o caratteri non validi come identificatore.

Gli oggetti possono contenere valori di qualsiasi tipo: primitivi, altri oggetti o funzioni (che in questo caso vengono chiamate *metodi*).

8.5.1 Costruzione

Un oggetto appena creato è inizialmente vuoto. È possibile arricchirlo dinamicamente durante l'esecuzione, semplicemente assegnando valori a nuove proprietà:

```
var o = new Object();
o.x = 7;
o.y = 9.5;
o["user_name"] = "Mario";
```

In JavaScript, l'aggiunta di una proprietà non richiede alcuna dichiarazione preventiva: l'oggetto si estende automaticamente.

8.5.2 Literal

È inoltre possibile creare e inizializzare un oggetto mediante la *notazione letterale*, una forma più compatta e comunemente utilizzata:

```
var o = { x: 7, y: 9.5, "user_name": "Mario" };
```

La notazione letterale consente di definire le proprietà direttamente al momento della creazione dell'oggetto e rappresenta la sintassi più diffusa nella programmazione JavaScript moderna.

8.5.3 Array

Gli array sono oggetti i cui elementi sono accessibili mediante indice numerico, a partire da 0. È possibile creare un nuovo array, eventualmente con dimensione iniziale prefissata, con:

```
var arr = new Array([dim]);
```

È resa poi disponibile la forma `literal`, ovvero: `[ ]`

8.6 Correlazione tra Oggetti e Array

Sebbene nella sintassi di JavaScript gli oggetti possano ricordare gli *array associativi*, essi non sono realmente array: gli oggetti sono strutture generiche composte da coppie **chiave** → **valore**, dove le chiavi sono stringhe (o simboli) e possono essere aggiunte dinamicamente.

Gli array, invece, sono oggetti specializzati ottimizzati per l'uso con indici numerici e forniscono metodi specifici (come `push()`, `pop()`, `sort()`, ecc.).

In altre parole:

- un **oggetto** è un contenitore generico di proprietà, accessibili tramite nome (stringa);
- un **array** è un oggetto indicizzato numericamente, con proprietà e comportamenti aggiuntivi pensati per gestire collezioni ordinate di elementi.

Per questo motivo, sebbene sia possibile accedere alle proprietà di un oggetto usando la notazione con parentesi quadre (come in un dizionario), oggetti e array hanno finalità e caratteristiche differenti, pur condividendo la stessa natura di base: tutto in JavaScript è un oggetto (tranne i tipi primitivi).

8.7 Stringhe

Le stringhe in JavaScript rappresentano sequenze arbitrarie di caratteri Unicode a 16 bit. Sono **immutabili**: una volta creata una stringa, il suo contenuto non può essere modificato; ogni operazione che sembra alterarla genera in realtà una nuova stringa.

Le stringhe possono essere espresse in forma *letterale* utilizzando:

- apici singoli: `'...'`;
- doppi apici: `"..."`;
- backtick: ``...``, per i *template literal* introdotti con ES6.

Esse supportano diverse operazioni:

- l'operatore `+`, che concatena stringhe oppure converte in stringa i valori concatenati;
- gli operatori di confronto `<`, `>`, `<=`, `>=`, che effettuano un confronto *lessicografico* basato sull'ordine Unicode;
- l'accesso ai singoli caratteri tramite la notazione con indice, ad esempio `s[0]`.

Sono inoltre disponibili numerosi metodi dell'oggetto **String**, come `length`, `substring()`, `indexOf()`, `toUpperCase()`, `replace()` e molti altri, utilizzabili per manipolare e analizzare i dati testuali (pur generando sempre nuove stringhe, data l'immutabilità del tipo).

8.7.1 Wrapper String

Sebbene le stringhe siano tipi primitivi, JavaScript mette a disposizione una classe **String** come *wrapper object*. Questo oggetto viene creato automaticamente dal motore JavaScript quando si invoca un metodo o una proprietà su una stringa primitiva.

Esempio:

```
var s = "ciao";
var len = s.length;      // il motore crea temporaneamente un oggetto String
var u   = s.toUpperCase();
```

L'oggetto wrapper:

- consente di accedere alle proprietà e ai metodi definiti in `String.prototype`;
- viene creato automaticamente quando necessario e distrutto immediatamente dopo l'uso;
- è distinto dalle stringhe oggetto create manualmente tramite `new String()`.

È infatti possibile creare esplicitamente un oggetto stringa:

```
var s1 = "test";           // stringa primitiva
var s2 = new String("test"); // oggetto String
```

Tuttavia, l'uso esplicito del costruttore `new String()` è **sconsigliato**, poiché:

- crea oggetti invece di valori primitivi;
- introduce comportamenti inattesi nei confronti, ad esempio:

```
"test" == new String("test")  // true (confronto per valore)
"test" === new String("test") // false (tipi diversi)
```

In sintesi, il wrapper **String** è utile per accedere ai metodi, ma non va mai istanziato manualmente.

8.7.2 Espressioni regolari

JavaScript offre le *regular expression* (o espressioni regolari) come tipo nativo di dato. Come per altri tipi complessi, esistono due modalità per creare una regex:

- **forma literal**, racchiusa tra `/ ... /`;
- **oggetto** `RegExp`, costruito dinamicamente.

Ad esempio:

```
// Forma literal
var r1 = /abc+/;

// Oggetto RegExp
var r2 = new RegExp("abc+");
```

Le espressioni regolari supportano inoltre vari **flag**, specificati dopo la barra di chiusura nella forma literal o come secondo argomento del costruttore:

- **g** — ricerca globale (non si ferma al primo match);
- **i** — case-insensitive;
- **m** — modalità multilinea;
- **u** — supporto completo Unicode;
- **s** — permette che il punto `.` corrisponda anche ai caratteri di nuova linea;
- **y** — ricerca “sticky”, a partire dalla posizione corrente.

Esempio con flag:

```
var r3 = /test/gi;
var r4 = new RegExp("test", "gi");
```

Le regex dispongono di metodi dedicati, tra cui:

- **test(str)** — restituisce `true` se la stringa rispetta il pattern;
- **exec(str)** — restituisce un array con il match trovato;
- utilizzo diretto nelle stringhe tramite metodi come `match()`, `search()`, `replace()`, `split()`.

Le espressioni regolari sono quindi parte integrante del linguaggio e rappresentano uno strumento fondamentale per l’analisi e la manipolazione avanzata delle stringhe.

8.8 Funzioni

Le funzioni in JavaScript, coerentemente con la natura dinamica del linguaggio, non richiedono la dichiarazione di un tipo di ritorno né di tipi per i parametri. Sono definite tramite la keyword **function** e possono accettare un numero arbitrario di argomenti.

```
function sum(x, y) {
    return x + y;
}

var s = sum(1, 2);
```

8.8.1 Literal e costruttori

È possibile creare funzioni anche tramite *function literal* o mediante il costruttore `Function()`. Una function literal consiste nell'assegnare direttamente una funzione anonima a una variabile:

```
var sum = function(x, y) {  
    return x + y;  
};
```

Il costruttore `Function`, invece, permette di creare funzioni dinamicamente a partire da stringhe, anche se tale tecnica è generalmente sconsigliata per motivi di prestazioni e sicurezza:

```
var sum = new Function("x", "y", "return x + y");
```

8.8.2 Parametri e argomenti

JavaScript non verifica la corrispondenza tra numero di parametri dichiarati e argomenti passati. Gli argomenti mancanti assumono valore `undefined`, mentre gli extra sono accessibili tramite l'oggetto speciale `arguments` (nelle funzioni non freccia).

8.8.3 Funzioni anonime e arrow function

Le versioni moderne del linguaggio (ES6) introducono le *arrow function*, una sintassi più compatta per definire funzioni:

```
const sum = (x, y) => x + y;
```

Le arrow function non hanno un proprio `this` e non possono essere usate come costruttori, differendo quindi semanticamente dalle funzioni tradizionali.

8.8.4 Metodi

In JavaScript un **metodo** è una funzione assegnata a una proprietà di un oggetto. Quando una funzione viene invocata come metodo, il valore della keyword `this` all'interno della funzione viene automaticamente impostato all'oggetto che la contiene.

```
var persona = {  
    nome: "Mario",  
    saluta: function() {  
        console.log("Ciao, sono " + this.nome);  
    }  
};
```

```
persona.saluta();    // "Ciao, sono Mario"
```

Il valore di `this` dipende dalla modalità di invocazione della funzione e non dalla sua posizione nel codice. Per questo motivo, è importante distinguere tra:

- invocazione come metodo: `obj.metodo()`, dove `this = obj`;
- invocazione come funzione semplice: `metodo()`, dove `this` in modalità *non strict* fa riferimento all'oggetto globale, mentre in *strict mode* vale `undefined`.

8.8.5 Costruttori

I **costruttori** sono funzioni utilizzate per creare nuovi oggetti. Una funzione diventa un costruttore quando viene invocata tramite la keyword **new**. In questo caso:

- viene creato automaticamente un nuovo oggetto vuoto;
- **this** all'interno della funzione fa riferimento all'oggetto in costruzione;
- l'oggetto creato eredita dal **prototype** della funzione costruttore;
- l'oggetto risultante viene restituito implicitamente (salvo ritorni espliciti di altri oggetti).

Esempio di costruttore:

```
function Punto(x, y) {  
    this.x = x;  
    this.y = y;  
    this.somma = function() {  
        return this.x + this.y;  
    };  
}
```

```
var p = new Punto(3, 4);  
console.log(p.somma()); // 7
```

È buona pratica scrivere i costruttori con l'iniziale maiuscola per distinguerli dalle funzioni ordinarie. I metodi comuni agli oggetti dello stesso tipo andrebbero preferibilmente definiti nel **prototype**, evitando di duplicare le funzioni in ogni istanza:

```
Punto.prototype.somma = function() {  
    return this.x + this.y;  
};
```

8.8.6 Metodi statici

JavaScript prevede l'esistenza di **metodi statici**, ovvero funzioni associate direttamente al costruttore (o alla classe) e non alle singole istanze dell'oggetto.

Un metodo è statico quando viene definito come proprietà della funzione costruttore, anziché del suo **prototype**. Di conseguenza, tali metodi sono invocabili tramite il nome del costruttore e non attraverso gli oggetti creati.

```
function Punto(x, y) {  
    this.x = x;  
    this.y = y;  
}  
  
// Metodo statico  
Punto.distanzaOrigine = function(p) {  
    return Math.sqrt(p.x * p.x + p.y * p.y);  
};  
  
var p = new Punto(3, 4);  
  
console.log(Punto.distanzaOrigine(p)); // 5  
console.log(p.distanzaOrigine);        // undefined
```

L'istanza **p** non può invocare il metodo statico.

8.9 Operatori

JavaScript supporta la maggior parte degli operatori presenti nei linguaggi della famiglia C (come C e Java), mantenendo le stesse regole di precedenza e associatività. Oltre agli operatori aritmetici, logici e di confronto standard, JavaScript introduce alcuni operatori aggiuntivi o con comportamento peculiare.

Tra le principali estensioni e differenze abbiamo:

- **delete**: rimuove una proprietà da un oggetto o un elemento da un array.
- **void**: valuta un'espressione e restituisce sempre **undefined**.
- **typeof**: restituisce il tipo di un'espressione come stringa.
- **===**: operatore di uguaglianza *stretta* (confronta valore e tipo senza conversioni implicite).
- **!==**: operatore di disuguaglianza *stretta*.

Gli operatori **===** e **!==** sono generalmente preferibili rispetto a **==** e **!=**, poiché questi ultimi applicano conversioni automatiche di tipo (*type coercion*) che possono generare comportamenti imprevisti.

Oltre a questi, JavaScript supporta operatori utili come:

- **in**: verifica se una proprietà è presente in un oggetto.
- **instanceof**: verifica se un oggetto deriva da un determinato costruttore.
- **new**: crea una nuova istanza a partire da un costruttore.
- **?** : operatore ternario.

8.10 Istruzioni

Le sequenze di istruzioni JavaScript seguono una sintassi molto simile a quella del linguaggio C. Ogni istruzione termina con un punto e virgola (spesso opzionale) e il linguaggio prevede diverse categorie di istruzioni, tra cui:

- **Espressioni**: istruzioni che valutano un'espressione (assegnamenti, chiamate di funzione, operazioni aritmetiche).
- **Istruzioni composte**: blocchi racchiusi tra { e }, che possono contenere una sequenza di altre istruzioni.
- **Istruzioni vuote**: una singola ; che non esegue alcuna operazione.
- **Istruzioni etichettate**: istruzioni precedute da un'etichetta seguita da :, usate soprattutto con **break** e **continue**.
- **Strutture di controllo**: istruzioni che alterano il flusso di esecuzione (condizionali, cicli, salti).
- **Definizioni e dichiarazioni**: dichiarazioni di variabili, funzioni, classi.
- **Istruzioni speciali**: istruzioni come **return**, **throw**, **break**, **continue**.

8.10.1 Istruzioni di controllo

JavaScript offre un insieme di strutture di controllo del flusso molto simili a quelle del C. Le principali sono:

- **if, else if, else**: selezione condizionale.
- **switch**: selezione multipla basata su uguaglianza stretta (**===**).
- **while**: ciclo basato su condizione pre-verificata.
- **do...while**: ciclo con condizione post-verificata.
- **for**: ciclo con inizializzazione, condizione e aggiornamento.
- **for...in**: iterazione sulle proprietà enumerabili di un oggetto.
- **for...of**: iterazione sugli elementi di un oggetto iterabile (array, stringhe, mappe, ecc.).
- **break**: interruzione di un ciclo o uscita da uno switch.
- **continue**: salta all'iterazione successiva del ciclo.
- **try...catch...finally**: gestione delle eccezioni.
- **throw**: sollevamento di un'eccezione.

8.11 Oggetto globale

In JavaScript tutte le variabili e funzioni dichiarate nello *scope globale* appartengono a un oggetto globale implicitamente definito. Il nome effettivo dell'oggetto globale dipende dal contesto di esecuzione:

- nei browser è l'oggetto `window`;
- negli script eseguiti con `Node.js` è `global`;
- negli ambienti moderni è accessibile tramite `globalThis`, un riferimento universale indipendente dalla piattaforma.

L'oggetto globale espone diverse funzioni utili, disponibili automaticamente in qualsiasi script senza necessità di importazione:

- `eval()`: interpreta ed esegue una stringa come codice JavaScript.
- `isFinite()`: restituisce `true` se il valore è un numero finito.
- `isNaN()`: restituisce `true` se il valore non è un numero (`NaN`).
- `parseInt()`: converte una stringa in un intero.
- `parseFloat()`: converte una stringa in un valore floating point.

In un browser, oltre a queste funzioni, l'oggetto globale `window` include molte altre funzionalità, come le API del DOM, i timer (`setTimeout`, `setInterval`), la gestione di finestre e molto altro.

8.12 JavaScript in HTML

Per inserire codice JavaScript all'interno di una pagina HTML si utilizza il tag dedicato:

```
<script>
  // codice JavaScript
</script>
```

Il tag `<script>` può contenere codice direttamente (script inline) oppure può fare riferimento a un file esterno tramite l'attributo `src`:

```
<script src="nomeFile.js"></script>
```

Quando è presente l'attributo `src`, il contenuto interno del tag viene ignorato. Storicamente era necessario specificare il tipo di script tramite l'attributo `type`, ad esempio:

```
<script type="text/javascript"> ... </script>
```

Nelle versioni moderne di HTML (HTML5) l'attributo `type` non è più richiesto, poiché JavaScript è considerato il linguaggio di default. Può comunque essere utilizzato per indicare moduli ECMAScript:

```
<script type="module" src="main.js"></script>
```

Gli script possono essere collocati sia nella sezione `<head>` che nel `<body>`, ma è pratica comune inserirli al termine del `<body>` per evitare il blocco del rendering della pagina. L'interpretazione dell'HTML, infatti, avviene in modo sequenziale: il browser interrompe l'analisi del documento per eseguire ogni script non marcato come asincrono.

Quando si utilizzano script interni, le variabili e le funzioni dichiarate negli script presenti all'interno dell'`<head>` diventano disponibili per tutto il resto del documento e possono essere utilizzate da qualsiasi altro script definito successivamente nella pagina, indipendentemente dalla loro posizione.

8.12.1 Disattivazione di JavaScript

Alcuni browser non supportano l'esecuzione di JavaScript oppure consentono all'utente di disattivarlo manualmente. In questi casi HTML mette a disposizione il tag `<noscript>`, il cui contenuto viene mostrato solo quando JavaScript è disabilitato o non disponibile. Può essere utilizzato per fornire messaggi informativi, alternative di navigazione o contenuti fallback.

8.13 Utilizzi di JavaScript

JavaScript può essere utilizzato principalmente per quattro scopi fondamentali:

- **Costruire dinamicamente parti di pagina:** generare o modificare contenuti HTML e CSS in tempo reale.
- **Rilevare informazioni sull'ambiente:** ottenere dati sul browser, sistema operativo, dimensioni della finestra, supporto per funzionalità, ecc.
- **Rispondere ad eventi:** gestire interazioni dell'utente come click, movimenti del mouse, input da tastiera, invio di form, e altri eventi.
- **Modificare dinamicamente il DOM:** aggiungere, rimuovere o aggiornare elementi, attributi e stili della pagina senza ricaricare il documento.

8.14 Browser Objects

Per interagire con la pagina e con l'ambiente del browser, JavaScript mette a disposizione una serie di oggetti predefiniti, detti **Browser Objects** e **DOM Objects**.

In particolare, la radice della gerarchia degli oggetti del documento è rappresentata dall'oggetto **document**, che corrisponde al *Document Object Model* (DOM). Attraverso **document** è possibile accedere, creare, modificare ed eliminare elementi della pagina, gestire eventi e interagire con l'albero degli elementi HTML in modo dinamico.

8.14.1 Costruzione dinamica della pagina

Per consentire la costruzione dinamica della pagina, l'oggetto **document** in JavaScript mette a disposizione diversi metodi, tra cui **write()** e **writeln()**. Questi metodi permettono di inserire contenuti HTML direttamente nel documento mentre viene interpretato dal browser.

Sono particolarmente utili per adattare le pagine in base al tipo di browser, alla risoluzione dello schermo o ad altre caratteristiche dell'ambiente client. Va notato, però, che l'uso eccessivo di **document.write()** è sconsigliato nelle applicazioni moderne, poiché può sovrascrivere l'intero documento se eseguito dopo il caricamento della pagina. Alternative più sicure e flessibili sono l'utilizzo del DOM con metodi come **createElement()** e **appendChild()**.

8.14.2 Rilevazione del browser

Per ottenere informazioni sul browser, JavaScript mette a disposizione l'oggetto **navigator**, che espone diverse proprietà utili:

- **appName:** nome interno del browser;
- **appName:** nome del browser;
- **appVersion:** versione del browser (nota: non sempre affidabile);
- **cookieEnabled:** indica se i cookie sono abilitati (**true** o **false**);
- **platform:** sistema operativo o piattaforma del client;
- **userAgent:** stringa identificativa del browser e della piattaforma.

8.14.3 Rilevazione delle proprietà dello schermo

Per accedere alle informazioni relative allo schermo del client, JavaScript fornisce l'oggetto **screen**, che include proprietà come:

- **screen.width:** larghezza totale dello schermo in pixel;
- **screen.height:** altezza totale dello schermo in pixel.

Queste proprietà possono essere utilizzate, ad esempio, per adattare il layout della pagina o la dimensione di elementi grafici in modo dinamico.

8.15 Modello ad eventi

JavaScript permette di associare una serie di eventi a funzioni di gestione, dette *event handler*. L'associazione tra eventi e handler può avvenire tramite attributi nei tag HTML o mediante registrazione dinamica con il DOM. Nei gestori di eventi è possibile intervenire sulla struttura della pagina, modificando il DOM (DHTML).

8.15.1 Eventi

Evento	Applicabilità	Occorrenza	EventHandler
abort	Immagini, audio, video	Utente blocca il caricamento	onAbort
blur	Finestre ed elementi del form	Utente rimuove il focus	onBlur
change	Input, select, textarea	Modifica del valore	onChange
click	Tutti gli elementi	Click del mouse	onClick
dragdrop	Elementi drag	Completamento drag-and-drop	onDragDrop
error	Immagini, script, media	Errore durante il caricamento	onError
focus	Input, textarea	Elemento riceve il focus	onFocus
keydown	Input, document	Pressione di un tasto	onKeyDown
keypress	Input, document	Pressione di un tasto generando carattere	onKeyPress
keyup	Input, document	Rilascio di un tasto	onKeyUp
load	Body, immagini, frame	Completamento caricamento	onLoad
mousedown	Tutti gli elementi	Premere un pulsante del mouse	onMouseDown
mousemove	Tutti gli elementi	Movimento del mouse	onMouseMove
mouseout	Tutti gli elementi	Uscita del mouse dall'elemento	onMouseOut
mouseover	Tutti gli elementi	Passaggio del mouse sopra l'elemento	onMouseOver
mouseup	Tutti gli elementi	Rilascio del pulsante del mouse	onMouseUp
move	Window	Movimento della finestra	onMove
reset	Form	Resettaggio di un form	onReset
resize	Window	Ridimensionamento della finestra	onResize
select	Input, textarea	Selezione del testo	onSelect
submit	Form	Invio del form	onSubmit
unload	Body, frame	Uscita dalla pagina	onUnload

8.15.2 Handlers

Per associare un *event handler* a un evento in JavaScript, è possibile utilizzare gli attributi dei tag HTML corrispondenti all'evento. La sintassi generale è:

```
<tag eventHandler="javascript_code">
```

Ad esempio, per eseguire una funzione al click di un pulsante:

```
<button onclick="saluta()">Cliccami</button>
```

```
<script>
function saluta() {
    alert("Ciao!");
}
</script>
```

In questo modo, quando l'utente clicca sul pulsante, viene eseguita la funzione `saluta()`.

Gli event handler possono essere associati a molti eventi, come `onClick`, `onLoad`, `onMouseOver`, ecc., e possono contenere direttamente codice JavaScript o chiamare funzioni definite nello script.

8.16 Esplorazione del DOM

Per esplorare il *DOM* (Document Object Model), JavaScript mette a disposizione l'oggetto `document`, che rappresenta l'intera struttura della pagina HTML. Questo oggetto espone diverse collezioni di elementi predefiniti:

- **anchors**: collezione di tutti i link con ancora (`<a name="...">`);
- **forms**: collezione di tutti i form presenti nella pagina;
- **images**: collezione di tutte le immagini (`<img>`);
- **links**: collezione di tutti i link (`<a href="...">`) esclusi quelli con solo ancora.

L'accesso agli elementi di queste collezioni può avvenire sia per indice numerico (ordine di apparizione nella pagina) sia tramite `id` o `name`.

8.16.1 Oggetto document

L'oggetto `document` fornisce numerose proprietà per interrogare lo stato della pagina:

- **bgColor**: colore di sfondo della pagina;
- **fgColor**: colore del testo della pagina;
- **lastModified**: data e ora dell'ultima modifica della pagina;
- **cookie**: stringa contenente i cookie della pagina;
- **title**: titolo della pagina;
- **URL**: URL corrente della pagina.

E una serie di metodi utili per manipolare o scrivere contenuti:

- **getElementById(id)**: restituisce l'elemento con l'id specificato;
- **write(string)**: scrive direttamente contenuto HTML nella pagina;
- **writeln(string)**: come `write`, ma aggiunge una nuova riga.

8.16.2 Oggetto Form

Ogni elemento `form` in JavaScript è rappresentato come un oggetto, che permette di accedere ai suoi campi tramite il vettore `elements[]`. Ogni elemento del form può, a sua volta, riferirsi al form padre tramite la proprietà `this.form`.

Proprietà e metodi comuni degli elementi del form

- **id**: identificatore unico dell'elemento;
- **form**: riferimento al form padre dell'elemento;
- **name**: nome dell'elemento, utilizzato per inviare i dati del form;
- **value**: valore corrente dell'elemento;
- **type**: tipo dell'elemento (`text`, `checkbox`, `radio`, ecc.);
- **className**: nome della classe CSS associata all'elemento;
- **disabled**: indica se l'elemento è disabilitato;
- **blur()**: rimuove il focus dall'elemento;
- **focus()**: assegna il focus all'elemento;
- **click()**: simula un click sull'elemento;
- **onblur**: event handler attivato quando l'elemento perde il focus;
- **onfocus**: event handler attivato quando l'elemento riceve il focus;
- **onclick**: event handler attivato al click sull'elemento.

Proprietà e metodi dell'oggetto form

- **action**: URL a cui inviare i dati del form;
- **elements**: vettore contenente tutti gli elementi del form;
- **length**: numero di elementi contenuti nel form;
- **method**: metodo di invio (GET o POST);
- **name**: nome del form;
- **target**: finestra o frame in cui inviare la risposta;
- **reset()**: resetta tutti i valori degli elementi ai valori iniziali;
- **submit()**: invia i dati del form;
- **onReset**: event handler eseguito al reset del form;
- **onSubmit**: event handler eseguito prima dell'invio del form.

8.17 Widget di un form

Ogni tipo di widget di un form HTML è rappresentato in JavaScript come un oggetto specifico, con proprietà e metodi dedicati. I principali widget includono:

- **Text**: `<input type="text">`, campo di testo a riga singola;
- **Checkbox**: `<input type="checkbox">`, casella di controllo selezionabile;
- **Radio**: `<input type="radio">`, pulsante di opzione all'interno di un gruppo;
- **Button**: `<input type="button">` o `<button>`, pulsante generico;
- **Hidden**: `<input type="hidden">`, campo nascosto non visibile all'utente;
- **File**: `<input type="file">`, campo per la selezione di file;
- **Password**: `<input type="password">`, campo di testo mascherato;
- **Textarea**: `<textarea>`, area di testo multilinea;
- **Submit**: `<input type="submit">`, pulsante per inviare il form;
- **Reset**: `<input type="reset">`, pulsante per resettare i valori del form ai valori iniziali.

8.17.1 Proprietà e metodi dell'oggetto Text

L'oggetto `Text` (corrispondente a `<input type="text">`) espone una serie di proprietà e metodi specifici oltre a quelli comuni a tutti gli elementi del form:

- **defaultValue**: valore iniziale dell'elemento;
- **disabled**: indica se l'elemento è disabilitato;
- **maxLength**: numero massimo di caratteri consentiti;
- **readOnly**: indica se l'elemento è di sola lettura;
- **size**: dimensione visuale dell'input (numero di caratteri visibili);
- **select()**: seleziona tutto il contenuto del campo di testo.

8.17.2 Proprietà dell'oggetto Checkbox/Radio

Gli oggetti `Checkbox` e `Radio` (corrispondenti a `<input type="checkbox">` e `<input type="radio">`) espongono le seguenti proprietà specifiche:

- **checked**: indica se l'elemento è attualmente selezionato;
- **defaultChecked**: indica lo stato di selezione iniziale dell'elemento.

8.18 Validazione del form

La validazione dei campi di un form è uno dei compiti principali di JavaScript lato client. Permette di ridurre il carico di lavoro del server, intercettando e bloccando preventivamente le richieste contenenti dati non validi.

Gli eventi tipicamente utilizzati per la validazione sono:

- **onChange**: viene eseguito durante la modifica di un campo, permettendo controlli immediati sul valore inserito;
- **onClick**: può essere usato per convalidare singoli pulsanti o azioni specifiche;
- **onSubmit**: eseguito prima dell'invio del form, consente di bloccare l'invio se uno o più campi non soddisfano i requisiti.

Tipicamente, le funzioni di validazione ritornano **true** se i dati sono validi e **false** se devono essere bloccati.

9 Ajax

9.1 Postback

Con l'introduzione del DHTML si è arrivati ad un modello ad eventi più simile a quello delle applicazioni tradizionali. In questo contesto si distinguono due principali tipologie di eventi:

- **Eventi locali:** eventi gestiti direttamente tramite JavaScript, che modificano il DOM senza richiedere comunicazioni con il server;
- **Eventi remoti:** eventi che richiedono una comunicazione con il server tramite richieste GET o POST, comportando il ricaricamento della pagina.

Il processo di ricaricare l'intera pagina per rispondere ad un evento prende il nome di **postback**.

9.1.1 Limiti

Il modello basato sul postback risulta spesso inadeguato rispetto al livello di interattività richiesto dalle moderne applicazioni Web. Ad ogni evento che richiede dati dal server, infatti, l'intero documento viene nuovamente caricato, seguendo il ciclo:

click → wait → refresh

Questo approccio presenta alcuni svantaggi:

- **Lentezza:** il ricaricamento completo della pagina introduce tempi di attesa percepibili dall'utente;
- **Scarso real-time:** l'interazione non è realmente immediata e fluida;
- **Spreco di risorse:** vengono trasferiti e ridisegnati contenuti identici, pur necessitando l'aggiornamento di una sola parte della pagina;
- **Perdita dello stato:** elementi dell'interfaccia (ad esempio campi compilati o posizioni di scorrimento) possono essere reimpostati dopo il refresh.

In sintesi, il postback rende l'interazione sequenziale e bloccante, ostacolando l'esperienza dinamica tipica delle applicazioni moderne.

9.2 Nascita Ajax

Il modello AJAX nasce per superare le limitazioni del postback, evitando il ricaricamento completo della pagina ad ogni interazione con il server. L'obiettivo è realizzare applicazioni Web più *user friendly* e altamente interattive (RIA — *Rich Internet Applications*), consentendo a JavaScript di comunicare con il server in modo selettivo e non bloccante.

Con AJAX, infatti, è possibile:

- inviare e ricevere dati dal server **senza** ricaricare l'intera pagina;
- aggiornare solo le parti del DOM interessate;
- migliorare la reattività e l'esperienza utente;
- ridurre il traffico di rete e il carico sul server.

L'elemento centrale di questa architettura è l'oggetto `XMLHttpRequest`, che permette a JavaScript di instaurare una comunicazione **asincrona** con il server utilizzando richieste HTTP (GET, POST, ecc.). Il server risponde restituendo dati — storicamente in formato XML, ma oggi spesso JSON — che possono essere elaborati e utilizzati per aggiornare dinamicamente la pagina.

9.2.1 Sequenza AJAX

Una tipica sequenza AJAX si articola nei seguenti passaggi fondamentali:

- **Interazione dell'utente:** l'utente compie un'azione sulla pagina Web (clic, modifica di un campo, selezione, ecc.).
- **Invocazione di una funzione JavaScript:** viene eseguita una funzione che:
 - crea un oggetto `XMLHttpRequest`;
 - definisce una funzione di callback per gestire la risposta;
 - invia una richiesta asincrona al server tramite HTTP (GET o POST).
- **Elaborazione della risposta:** il server elabora la richiesta e restituisce un risultato (XML, JSON, testo, ecc.).
- **Callback e aggiornamento del DOM:** la funzione di callback viene invocata automaticamente al completamento della richiesta e aggiorna dinamicamente la pagina modificando il DOM, senza ricaricare l'intero documento.

9.3 XMLHttpRequest

L'oggetto `XMLHttpRequest` è il componente base del modello AJAX e permette a JavaScript di effettuare chiamate HTTP verso un server Web senza ricaricare l'intera pagina.

Supporta i principali metodi di invio delle richieste, tra cui **GET** e **POST**. È possibile, poi, inviare richieste sia in modalità sincrona che asincrona:

- **Sincrona:** il codice rimane bloccato in attesa della risposta del server. L'esecuzione prosegue solo al completamento della richiesta.
- **Asincrona:** la richiesta viene inviata senza interrompere il flusso di esecuzione del programma. La risposta viene gestita tramite una funzione di callback associata all'evento `onreadystatechange`. È la modalità standard e preferita in AJAX.

In modalità asincrona l'esecuzione non viene bloccata: non viene avviato un "thread separato", ma la risposta è gestita dal browser tramite il suo event loop e notificata quando disponibile.

9.3.1 Creazione

Nella maggior parte dei browser moderni l'oggetto `XMLHttpRequest` è supportato nativamente e può essere creato semplicemente tramite:

```
var xhr = new XMLHttpRequest();
```

Nei browser più datati (come le vecchie versioni di Internet Explorer) era necessario ricorrere ad un oggetto `ActiveXObject`, ma questa soluzione è oggi considerata obsoleta.

Per motivi di sicurezza, l'utilizzo di `XMLHttpRequest` è soggetto alla **Same-Origin Policy**, secondo la quale una pagina Web può effettuare richieste solo verso la *stessa origine* che ha fornito il documento, ovvero lo stesso:

- protocollo (es. `http` / `https`)
- dominio
- porta

Questo impedisce l'invio di richieste AJAX verso server esterni non autorizzati.

L'accesso a risorse di diversa origine è consentito solo se il server remoto abilita esplicitamente la comunicazione tramite il meccanismo **CORS** (Cross-Origin Resource Sharing).

9.3.2 Metodi

L'oggetto `XMLHttpRequest` mette a disposizione diversi metodi per la gestione delle richieste HTTP. Sebbene il supporto possa variare tra browser, è buona pratica basarsi sul sottoinsieme comune e maggiormente compatibile (storicamente quello supportato da Safari).

open() Inizializza la richiesta verso il server, specificando metodo, URL e modalità asincrona:

```
open(method, url [, async] [, user] [, password])
```

- **method**: "GET", "POST", ecc.
- **url**: indirizzo della risorsa da chiamare
- **async** (opzionale): **true** (default) per richiesta asincrona, **false** per sincrona
- **user**, **password** (opzionali): credenziali per autenticazione HTTP

La chiamata a **open()** non invia la richiesta, ma si limita a configurarla.

setRequestHeader() Permette di impostare gli header HTTP della richiesta. Può essere invocato solo dopo **open()** e prima di **send()**.

```
setRequestHeader(header, value)
```

Esempio comune (per richieste POST con dati di form):

```
xhr.setRequestHeader("Content-Type",  
    "application/x-www-form-urlencoded");
```

send() Invia la richiesta al server:

```
send([body])
```

- Se il metodo è GET, **body** deve essere omesso o **null**
- Se il metodo è POST, **body** contiene i dati da inviare

Se la richiesta è asincrona (**async = true**), **send()** **non blocca** il flusso di esecuzione. In caso di richiesta sincrona (**async = false**), l'esecuzione rimane bloccata fino alla risposta del server.

Altri metodi utili

- **abort()**: interrompe una richiesta in corso
- **getAllResponseHeaders()**: restituisce tutti gli header di risposta
- **getResponseHeader(header)**: restituisce il valore di un singolo header di risposta

9.3.3 Proprietà

L'oggetto **XMLHttpRequest** mette a disposizione diverse proprietà attraverso le quali è possibile conoscere lo stato della richiesta e leggerne il risultato. I dati restituiti dal server non vengono inviati direttamente alla funzione chiamante, ma sono accessibili tramite queste proprietà.

readyState Indica lo stato corrente della richiesta. Assume valori interi da 0 a 4:

- **0** – *UNSENT*: oggetto creato, ma richiesta non inizializzata
- **1** – *OPENED*: metodo e URL impostati tramite **open()**
- **2** – *HEADERS_RECEIVED*: sono disponibili gli header della risposta
- **3** – *LOADING*: la risposta è in fase di trasferimento
- **4** – *DONE*: richiesta completata (con successo o errore)

Il valore **4** è l'unico garantito in modo uniforme da tutti i browser e rappresenta il momento in cui è possibile leggere il risultato della richiesta.

onreadystatechange Rappresenta la funzione di callback invocata ogni volta che cambia il valore di **readyState**. È qui che normalmente si intercetta lo stato 4 per gestire la risposta:

```
xhr.onreadystatechange = function() {  
    if (xhr.readyState == 4) {  
        // gestione risposta  
    }  
}
```

status Contiene il codice di stato HTTP restituito dal server (ad esempio 200, 404, 500...). Un valore tipico di successo è **200**.

La proprietà **statusText** restituisce la corrispondente descrizione testuale (es. "OK", "Not Found").

responseText Contiene la risposta del server in formato testo (stringa). È la proprietà più utilizzata, soprattutto quando il server restituisce dati in formato HTML, JSON o testo semplice.

responseXML Se la risposta del server è un contenuto XML valido e il browser è in grado di interpretarlo, essa viene automaticamente convertita in un documento XML DOM e resa disponibile in **responseXML**. In caso contrario, vale **null**.

9.3.4 Funzione di callback

La funzione di callback associata alla proprietà **onreadystatechange** viene eseguita automaticamente ogni volta che il valore di **readyState** cambia. Questo significa che può essere invocata più volte durante l'elaborazione della richiesta, fino al completamento (stato 4).

All'interno della callback è possibile accedere a tutte le proprietà dell'oggetto **XMLHttpRequest**, comprese:

- gli **header** della risposta tramite **getResponseHeader()** o **getAllResponseHeaders()**
- il **codice di stato HTTP** tramite **status**
- il **contenuto della risposta** tramite **responseText** o **responseXML**

Poiché solo lo stato 4 (*DONE*) indica che la richiesta è terminata, è buona norma controllare sia **readyState** che **status** prima di utilizzare la risposta:

```
xhr.onreadystatechange = function() {  
    if (xhr.readyState == 4 && xhr.status == 200) {  
        // utilizzo della risposta  
        var data = xhr.responseText;  
    }  
};
```

In questo modo si evita di elaborare dati incompleti o risultati derivanti da errori di rete o di server.

9.4 Vantaggi e svantaggi

L'utilizzo di Ajax introduce una serie di benefici ma anche alcune criticità legate al modello asincrono.

9.4.1 Vantaggi

- **Maggiore interattività:** la pagina non deve essere completamente ricaricata ad ogni azione dell'utente.
- **Migliore usabilità:** le applicazioni risultano più fluide e reattive, in stile *Rich Internet Applications (RIA)*.
- **Riduzione del traffico di rete:** vengono scambiati solo i dati necessari, non l'intera pagina HTML.
- **Minore carico sul server:** si evitano continui postback e rendering completi.
- **Aggiornamenti mirati del DOM:** è possibile modificare solo porzioni della pagina.

9.4.2 Svantaggi

- **Minore percezione dell'elaborazione:** l'utente potrebbe non rendersi conto che una richiesta è in corso, poiché non avviene un refresh della pagina.
- **Gestione complessa dell'asincronia:** il programmatore deve coordinare callback, stato dell'applicazione e possibili richieste concorrenti.
- **Maggiore difficoltà di debugging:** il flusso di esecuzione non è lineare e può risultare più difficile da tracciare.
- **Dipendenza dal JavaScript:** se disabilitato, l'applicazione potrebbe non funzionare correttamente.
- **Questioni di sicurezza e cache:** la comunicazione diretta con il server richiede attenzione a validazione e gestione dei dati.

In sintesi, con Ajax si guadagna in **interattività ed efficienza**, ma si introduce un modello di elaborazione **asincrono** che richiede maggiore attenzione sia nella progettazione che nella gestione dell'esperienza utente.

9.5 Criticità di comunicazione

Nel modello asincrono di Ajax l'utente potrebbe non essere consapevole che un'operazione è in corso, poiché l'interfaccia non viene ricaricata. Questo può portare a comportamenti indesiderati, come ripetere più volte la stessa azione o abbandonare prematuramente l'interazione.

Per ridurre tali problematiche possono essere adottate due tecniche:

- **implementazione di un sistema di caricamento:** indicatori visivi o messaggi che segnalano l'elaborazione in corso;
- **limitazione del tempo di risposta:** definizione di un timeout oltre il quale la richiesta viene interrotta.

9.5.1 Metodo `abort()`

L'oggetto `XMLHttpRequest` mette a disposizione il metodo `abort()`, che permette di interrompere una richiesta HTTP precedentemente avviata.

Questo risulta utile, ad esempio, quando:

- la risposta del server impiega troppo tempo;
- l'utente avvia una nuova richiesta che rende obsoleta la precedente;
- si desidera evitare ritardi o sovrapposizioni di elaborazioni.

Spesso il metodo `abort()` viene utilizzato in combinazione con `setTimeout()`, in modo da interrompere automaticamente la richiesta dopo un intervallo di tempo prestabilito. In tal caso, se la risposta non arriva entro il limite fissato, la funzione collegata a `abort()` viene eseguita, notificando eventualmente l'utente dell'errore o del timeout.

9.6 Gestione della risposta

I dati scambiati tra client e server possono essere restituiti in diversi formati. Uno dei formati storicamente più utilizzati con Ajax è l'XML. In questo caso, la risposta del server viene interpretata come un documento XML e può essere elaborata direttamente dal client.

Quando la risposta è in formato XML, l'oggetto `XMLHttpRequest` mette a disposizione la proprietà `responseXML`, che contiene il documento già parsificato. Tale documento può essere analizzato e manipolato utilizzando le API del **W3C DOM** (Document Object Model), le quali permettono di:

- navigare tra i nodi del documento XML;
- accedere a tag, attributi e valori testuali;
- ricercare elementi tramite metodi come `getElementsByTagName()`;
- estrarre informazioni utili per aggiornare dinamicamente il contenuto della pagina.

L'elaborazione del documento XML consente quindi di aggiornare il DOM della pagina Web senza ricaricare l'intera vista, mantenendo la logica di interazione asincrona tipica di Ajax.

9.7 JSON

Con JSON si indica la **JavaScript Object Notation**, un formato testuale per lo scambio di dati tra client e server. Rappresenta un'alternativa ad XML ed è oggi considerato il nuovo standard di fatto nelle comunicazioni Ajax.

JSON è particolarmente vantaggioso perché:

- è molto leggero in termini di dimensione dei dati scambiati;
- è semplice da leggere e scrivere;
- è estremamente efficiente da elaborare in JavaScript;
- può essere convertito direttamente in oggetti JavaScript senza parsing complesso.

La struttura di JSON si basa sugli **object literal** e **array literal** già presenti in JavaScript. I dati sono rappresentati tramite coppie **chiave: valore** racchiuse tra parentesi graffe, mentre gli insiemi ordinati di valori sono rappresentati tramite array.

Grazie alla sua semplicità e immediatezza di utilizzo, JSON è oggi il formato preferito per le comunicazioni asincrone tra applicazioni Web e server.

9.7.1 Da stringa ad oggetto

In JavaScript è possibile convertire una stringa JSON in un oggetto utilizzabile tramite la funzione `eval`. Per motivi di correttezza sintattica, l'argomento passato a `eval` deve essere racchiuso tra parentesi tonde, in modo da essere interpretato come un'espressione.

Esempio:

```
var jsonString = '{"nome": "Mario", "eta": 30}';
var obj = eval('(' + jsonString + ')');
console.log(obj.nome); // stampa "Mario"
```

L'uso di `eval` può comportare rischi di sicurezza, però, se la stringa non è controllata. Nelle versioni moderne di JavaScript è preferibile usare `JSON.parse(jsonString)` per ottenere lo stesso risultato in modo sicuro, prevenendo l'inserimento di codice malevolo.

9.7.2 Java

Anche in Java è possibile interpretare dati JSON tramite librerie apposite. Il parser più diffuso è **Gson**, sviluppato da Google, che permette di convertire stringhe JSON in oggetti Java e viceversa.

Esempio di utilizzo di Gson:

```
import com.google.gson.Gson;

Gson gson = new Gson();

// Da JSON a oggetto Java
String jsonString = "{\"nome\":\"Mario\",\"eta\":30}";
Persona p = gson.fromJson(jsonString, Persona.class);

// Da oggetto Java a JSON
String jsonOut = gson.toJson(p);
```

In questo modo, JSON diventa facilmente integrabile nelle applicazioni Java per scambio dati asincrono o persistente.

10 React

React.js è una **libreria JavaScript** (e non un framework) sviluppata da Facebook, nata per semplificare lo sviluppo di interfacce Web front-end interattive e dinamiche.

Si basa sul concetto di componenti riutilizzabili e sull'utilizzo del **DOM virtuale** per ottimizzare gli aggiornamenti dell'interfaccia utente. React viene comunemente utilizzato insieme a HTML5, CSS e JavaScript moderno.

10.1 Approccio

React.js nasce per facilitare lo sviluppo di **Single Page Applications (SPA)**, ovvero applicazioni Web basate su un'unica pagina HTML principale. In una SPA il browser:

- non ricarica completamente la pagina ad ogni interazione;
- aggiorna dinamicamente solo le parti necessarie dell'interfaccia;
- utilizza i dati ricevuti dal back-end (tipicamente tramite API).

In questo modello la pagina diventa un **contenitore dinamico** che evolve nel tempo, offrendo un'esperienza più fluida e simile a quella delle applicazioni desktop. Lo sviluppo in React avviene attraverso l'uso di **componenti**, ovvero unità logiche e riutilizzabili che:

- definiscono una parte dell'interfaccia utente;
- possono contenere logica e stato;
- possono essere combinati per formare interfacce complesse;
- comunicano tramite proprietà (**props**) e stato (**state**).

Ogni componente interagisce con le API di React e non manipola direttamente il DOM, ma ne *describe* la struttura e il comportamento. Sarà poi React ad aggiornare il DOM reale nel modo più efficiente possibile.

10.1.1 Virtual DOM

Con React.js viene introdotto il concetto di **Virtual DOM**, ovvero una copia del DOM mantenuta in memoria. Ogni volta che deve essere effettuata una modifica, React modifica il Virtual DOM e non quello reale.

Effettuata la modifica, viene poi eseguito un confronto tra il Virtual DOM aggiornato e quello precedente per verificare se sono presenti differenze grafiche. Se tali differenze esistono, viene aggiornato il DOM reale applicando solo le modifiche necessarie.

10.2 Vantaggi

I vantaggi di React sono sia per gli sviluppatori che per gli utilizzatori.

Sviluppatori

- L'approccio a componenti rende il codice riutilizzabile e permette di seguire il principio *write once*, favorendo la modularizzazione.
- La gestione del Virtual DOM è completamente a carico della libreria: lo sviluppatore deve concentrarsi unicamente sulla logica dei componenti.

Utilizzatori

- Alleggerimento e velocizzazione del rendering grazie all'utilizzo del Virtual DOM.

10.3 Element

Un React **Element** è un oggetto semplice e immutabile che descrive ciò che si vuole visualizzare sullo schermo. Solitamente rappresenta un nodo, che può contenere a sua volta istanze di altri nodi o componenti.

10.3.1 Esempio

```
<!DOCTYPE html>
<html>
  <head>
    <meta charset="UTF-8" />
    <title>Primi passi con React</title>
    <script src="https://unpkg.com/react@15/dist/react.js"></script>
    <script src="https://unpkg.com/react-dom@15/dist/react-dom.js"></script>
    <script src="https://cdnjs.cloudflare.com/ajax/libs/babel-core/5.8.24/browser.js"></script>
  </head>
  <body>
    <div id="root"></div>

    <script type="text/babel">
      class HelloWorld extends React.Component {
        render() {
          return <p>Hello</p>
        }
      }

      ReactDOM.render(<HelloWorld/>, document.getElementById('root'));
    </script>
  </body>
</html>
```

10.4 JSX

Il **JSX** (Javascript XML) è un'estensione sintattica di JavaScript utilizzata da React per descrivere l'interfaccia utente in una forma molto simile all'HTML. Consente di combinare struttura (markup) e logica (JavaScript) in modo semplice e leggibile, rendendo il codice più vicino alla rappresentazione reale del componente grafico.

A differenza dell'HTML tradizionale, il JSX permette di:

- inserire espressioni JavaScript all'interno del markup racchiudendole tra parentesi graffe {};
- comporre interfacce attraverso componenti riutilizzabili;
- applicare dinamicamente proprietà, condizioni e cicli;
- descrivere il contenuto da renderizzare in modo dichiarativo.

Il JSX non può essere eseguito direttamente dal browser: esso viene **trasformato** da un compilatore (come Babel) in normali chiamate JavaScript a `React.createElement()`, così da poter essere interpretato correttamente.

L'utilizzo di JSX non è obbligatorio, ma è fortemente consigliato perché:

- migliora la leggibilità del codice;
- riduce la verbosità rispetto alla creazione manuale di elementi;
- facilita la progettazione di componenti complessi.

React mette comunque a disposizione funzioni per creare elementi senza JSX, ma questa soluzione è solitamente riservata a casi particolari o strumenti di build automatizzati.

10.4.1 Variabili

All'interno di JSX è possibile utilizzare variabili JavaScript per determinare dinamicamente il contenuto visualizzato. Le variabili vengono dichiarate in JavaScript (ad esempio con `let` o `var`) e possono essere inserite nel JSX racchiudendole tra parentesi graffe {}.

```
let nome = "Mario";
return <p>Ciao {nome}</p>;
```

Le variabili possono contenere valori primitivi, oggetti o anche altri elementi React.

10.4.2 Costanti

Come per le variabili, anche le costanti dichiarate con `const` possono essere utilizzate nel JSX. La dichiarazione con `const` garantisce che il riferimento non possa essere riassegnato, rendendo il codice più sicuro e prevedibile.

```
const titolo = "Benvenuto!";
return <h1>{titolo}</h1>;
```

10.4.3 Funzioni

Nel JSX è possibile utilizzare funzioni JavaScript per calcolare valori o generare contenuti da visualizzare. Una funzione può essere invocata direttamente all'interno delle parentesi graffe:

```
function saluta(nome) {
  return "Ciao " + nome;
}

return <p>{saluta("Anna")}</p>;
```

Le funzioni possono anche restituire elementi React, rendendole utili per suddividere parti dell'interfaccia in modo modulare:

```
function Messaggio() {
  return <span>Messaggio di prova</span>;
}

return <div>{Messaggio()}</div>;
```

10.4.4 Compilatore

Il JSX non è supportato nativamente dai browser: nessun browser è in grado di interpretare direttamente codice JSX. Per questo motivo è necessario un processo di trasformazione che converta il JSX in normale JavaScript.

A tale scopo viene utilizzato un pre-compilatore chiamato `Babel`, il cui compito è tradurre il codice JSX in chiamate JavaScript equivalenti (ad esempio `React.createElement()`), rendendolo eseguibile dal browser. Nel caso di sviluppo semplice lato client, Babel può essere incluso direttamente nella pagina HTML:

```
<script src="https://cdnjs.cloudflare.com/ajax/libs/babel-core/5.8.24/browser.js"></script>
```

Inoltre, gli script JSX devono essere dichiarati con il tipo:

```
<script type="text/babel">
  // codice JSX
</script>
```

Nello sviluppo professionale, Babel viene eseguito in fase di build (ad esempio tramite Webpack, Vite o simili), evitando di compilare il JSX direttamente nel browser e migliorando prestazioni e sicurezza.

10.5 Componenti

I **React Component** sono le unità fondamentali di un'applicazione React. Rappresentano parti autonome e riutilizzabili dell'interfaccia utente e sono responsabili di descrivere ciò che deve essere visualizzato sullo schermo.

Un componente è dinamico: può ricevere input esterni e, in base a tali dati, determinare l'aspetto grafico da restituire. Gli input ricevuti sono detti **props** (properties) e vengono passati al componente dall'esterno, rimanendo immutabili all'interno del componente stesso.

Ogni componente, una volta eseguito, **restituisce elementi React** che descrivono l'interfaccia da rendere nel DOM.

In sintesi, un componente:

- incapsula una parte dell'interfaccia;
- può ricevere dati tramite **props**;
- restituisce elementi React da visualizzare;
- può essere riutilizzato all'interno dell'applicazione.

I nomi dei componenti devono sempre iniziare con una lettera maiuscola.

10.5.1 Tipi

Esistono principalmente due tipi di componenti React:

- **Componenti a funzione** (Function Components);
- **Componenti a classe** (Class Components).

Entrambi restituiscono una rappresentazione dell'interfaccia tramite una **return** contenente JSX.

Tuttavia, i componenti a classe possiedono funzionalità aggiuntive rispetto a quelli a funzione tradizionali, come l'accesso diretto allo **state** e ai metodi del ciclo di vita del componente.

Nelle versioni moderne di React, grazie agli **Hooks**, anche i componenti a funzione possono gestire stato e comportamenti complessi, diventando il modello consigliato per lo sviluppo.

10.5.2 Classi in linea

React permette anche la definizione di componenti tramite classi in linea usando il metodo `React.createClass`. Questo approccio era comune nelle versioni precedenti di React (ora deprecato in favore delle classi ES6 o dei componenti funzionali).

Esempio:

```
var Car = React.createClass({
  render: function() {
    return <p>Hello</p>;
  }
});
```

10.6 Props

Per trasferire dati ai componenti React si utilizza il concetto di **props** (properties). Le **props** sono valori immutabili passati dall'esterno al componente e servono principalmente a configurarne il comportamento o l'aspetto.

In fase di rendering, è possibile impostare le **props** come se fossero attributi HTML. Esempio:

```
<MyComponent title="Benvenuto" count={5} />
```

All'interno del componente, le **props** vengono lette tramite l'oggetto `this.props` (per i componenti a classe) o come argomento della funzione (per i componenti a funzione):

```
function MyComponent(props) {
  return <h1>{props.title}</h1>;
}

class MyComponent extends React.Component {
  render() {
    return <h1>{this.props.title}</h1>;
  }
}
```

10.7 State

In React, tutti i componenti di tipo classe possono gestire uno **state**, ovvero un insieme di proprietà interne al componente che possono variare nel tempo.

Le proprietà definite nello **state** non sono immutabili, a differenza delle **props**, e sono pensate per rappresentare lo stato interno del componente durante la sua vita.

Quando una proprietà dello **state** viene aggiornata tramite **this.setState()**, React esegue automaticamente il **rerendering** del componente, aggiornando la parte del DOM relativa a quel componente in modo efficiente.

10.7.1 Costruttore

Nei componenti di tipo classe, il **costruttore** viene utilizzato per inizializzare lo **state** del componente. Viene invocato prima del primo rendering e permette di impostare valori di default per le proprietà dello **state**.

Tipicamente, all'interno del costruttore si chiama anche il costruttore della superclasse tramite **super(props)** (necessario per accedere ai dati in **this**):

```
class MyComponent extends React.Component {
  constructor(props) {
    super(props);
    this.state = {
      count: 0,
      text: ''
    };
  }
  render() {
    return <p>{this.state.count}</p>;
  }
}
```

10.7.2 Aggiornamento

Per modificare lo **state** di un componente React, si utilizza il metodo **setState**, ereditato da **React.Component**. Ogni volta che **setState** viene chiamato, React effettua automaticamente un re-rendering del componente interessato, aggiornando l'interfaccia in base ai nuovi valori dello **state**.

È importante notare che:

- Lo **state** è *privato* per ciascun componente; nessun componente può accedere direttamente allo **state** di un altro componente.
- Le modifiche allo **state** devono essere effettuate solo tramite **setState** e non modificando direttamente l'oggetto **state**.

```
this.setState({ count: this.state.count + 1 });
```

10.7.3 Considerazioni

Non tutti i componenti React necessitano di uno **state**. Nella maggior parte dei casi, i componenti dovrebbero ricevere i dati dai componenti genitori sotto forma di **props**, mantenendo lo stato dell'applicazione gestito in modo centralizzato e gerarchico nei livelli più alti dell'albero dei componenti.

Questo approccio favorisce:

- una migliore separazione delle responsabilità;
- una maggiore riusabilità dei componenti;
- una struttura dell'applicazione più prevedibile e più semplice da mantenere.

I componenti privi di stato sono spesso detti *presentational components*, mentre quelli che gestiscono lo stato vengono definiti *container components*.

10.8 Gestione degli eventi

In React, la gestione degli eventi differisce leggermente dal JavaScript “classico”. In JSX, l'associazione di un *event handler* avviene tramite una sintassi simile a quella HTML, ma con alcune differenze fondamentali. Esempio:

```
<button onClick={handler}>  
  Bottone  
</button>
```

In particolare:

- i nomi degli eventi utilizzano la **camelCase** (ad esempio **onClick** invece di **onclick**);
- il valore dell'attributo non è una stringa, ma un **riferimento a una funzione JavaScript**;
- non viene utilizzato **addEventListener**: l'associazione è gestita internamente da React.

Questo approccio rende il codice più leggibile e mantiene la logica degli eventi strettamente legata al componente che la utilizza.

10.8.1 Eventi React

In React, ad ogni *event handler* viene passato un oggetto evento detto **Synthetic Event**. I *synthetic events* sono un'astrazione degli eventi nativi del browser, definiti secondo le specifiche del W3C, al fine di garantire un comportamento coerente e compatibile su tutti i browser.

Sebbene espongano un'interfaccia simile agli eventi nativi, i *synthetic events* non sono gli eventi originali del DOM: React li gestisce tramite un proprio sistema di *event delegation* per migliorare le prestazioni.

10.8.2 Accesso allo state

Per accedere allo **state** di un componente all'interno di un event handler è necessario garantire che il riferimento a **this** sia correttamente associato all'istanza del componente.

Questo può essere ottenuto in due modi principali:

- forzare il **bind** dell'oggetto **this** all'handler;
- definire e invocare l'handler come *arrow function*, che eredita automaticamente il contesto di **this**.

Binding

```
class Component extends React.Component {  
  constructor(props) {  
    super(props);  
    this.state = { var: value };  
  }  
}
```

```

        // binding esplicito dell'handler
        this.handleClick = this.handleClick.bind(this);
    }

    handleClick() {
        this.setState({ var: newValue });
    }

    render() {
        return (
            <input
                type="button"
                onClick={this.handleClick} // uso del bind
            />
        );
    }
}

```

Arrow function

```

class Component extends React.Component {
    state = { var: value };

    handleClick = () => {
        this.setState({ var: newValue });
    }

    render() {
        return (
            <input
                type="button"
                onClick={this.handleClick} // arrow function
            />
        );
    }
}

```

10.9 Form

In React i form funzionano in modo diverso rispetto all'HTML tradizionale. Negli elementi HTML standard (`input`, `textarea`, `select`) lo stato è gestito direttamente dal browser: ogni campo mantiene autonomamente il proprio valore.

In React, invece, è pratica comune **controllare esplicitamente lo stato dei form** attraverso lo **state** del componente. In questo modo, il valore dei campi non è più gestito dal DOM, ma dall'applicazione React stessa.

10.9.1 Invocazione risorse server

In React è possibile effettuare l'invocazione di risorse server utilizzando le **Fetch API** fornite da JavaScript per la comunicazione HTTP.

Le Fetch API permettono di inviare richieste asincrone (`GET`, `POST`, ecc.) verso il server e di gestire le risposte tramite *promise*, senza bloccare il flusso di esecuzione dell'applicazione.

Unendo questa API all'interfaccia **FormData** è possibile gestire in modo semplice ed efficiente l'invio dei dati dei form verso le risorse server, inclusi file e campi multipart, mantenendo l'interazione fluida e reattiva.

```

handleSubmit(event) {
  event.preventDefault();
  const data = new FormData(event.target);

  fetch("/api/endpoint", {
    method: "post",
    body: data
  });
}

return (
  <form onSubmit={this.handleSubmit}>
    ...
  </form>
)

```

10.10 Librerie alternative

Nel panorama dello sviluppo web esistono numerose librerie e framework alternativi a React, ciascuno con caratteristiche e filosofie differenti.

10.10.1 JQuery

jQuery è una libreria JavaScript nata per semplificare la manipolazione del DOM attraverso l'utilizzo dei selettori CSS3, la gestione degli eventi e le chiamate Ajax.

È compatibile con la maggior parte dei browser ed è stata a lungo un componente essenziale dei più diffusi CMS. Presenta inoltre un'estensione, detta **jQuery UI**, pensata per lo sviluppo di componenti grafici ed effetti visivi interattivi.

10.10.2 Angular

Angular è un framework completo sviluppato da Google, basato su TypeScript. Adotta un'architettura MVC/MVVM e fornisce nativamente strumenti per il routing, la gestione dello stato, la dependency injection e la comunicazione HTTP.

Angular integra una versione ridotta di jQuery, detta **jQuery Lite**, e permette lo sviluppo di applicazioni multipiattaforma. Attualmente rappresenta uno dei principali competitor di React nel panorama dello sviluppo front-end.

10.10.3 Vue

Vue.js è un framework progressivo per la costruzione di interfacce utente. Combina concetti di React e Angular, offrendo un approccio reattivo, component-based e semplice da integrare anche in applicazioni esistenti.

È meno complesso di Angular e più strutturato di React, risultando particolarmente adatto a progetti di media dimensione e per i principianti.

10.10.4 Meteor

Meteor è una piattaforma full-stack JavaScript per lo sviluppo di applicazioni web e mobile. Integra client, server e database in un unico ambiente di sviluppo, semplificando la sincronizzazione dei dati in tempo reale. Pur essendo meno flessibile rispetto a React, consente uno sviluppo rapido di applicazioni reattive.

Meteor si basa sul protocollo DDP (*Distributed Data Protocol*), che facilita l'aggiornamento automatico dei client, ed è costruito su Node.js, sfruttando la sua natura asincrona e orientata agli eventi.

10.10.5 Backbone

Backbone.js è una libreria JavaScript leggera che fornisce una struttura minimale per applicazioni web, basata su modelli, viste e collezioni.

Non gestisce direttamente il rendering del DOM e necessita dell'integrazione con altre librerie, come jQuery o Underscore.js. È stata originariamente concepita per lo sviluppo di *single-page applications* seguendo il pattern Model-View-Presenter. Oggi, tuttavia, Backbone è in gran parte superata da framework moderni come React, Angular e Vue.

11 Tecnologie e componenti server-side

11.1 Java Model 2

A differenza del Model 1, nel quale la logica applicativa risulta mescolata alla rappresentazione dei dati, il Java Model 2 prevede una chiara separazione tra le responsabilità di controllo e di presentazione già in fase progettuale.

Il Model 2 è comunemente associato al paradigma MVC (Model–View–Controller), anche se non ne rappresenta un'implementazione rigorosa e formale.

In questo modello:

- il **Controller** è generalmente implementato tramite **Servlet**;
- la **View** è realizzata tramite **JSP**;
- il **Model** non è rigidamente definito e può essere costituito da classi Java, JavaBeans o componenti di accesso ai dati.

11.2 Model-View-Controller

- **Model**: rappresenta i dati dell'applicazione e le operazioni di accesso e modifica su di essi. Ha il compito di mantenere le informazioni, gestirne la coerenza e notificare le *view* in caso di modifica. Deve rendere disponibili i dati alle *view* e le operazioni applicative ai *controller*.
- **View**: accede al *model* e ne specifica la rappresentazione grafica. Trasferisce l'input dell'utente al *controller* e si aggiorna automaticamente quando il *model* cambia stato.
- **Controller**: gestisce le richieste dell'utente, eseguendo il *dispatching* delle azioni, mappandole sulle operazioni applicative e selezionando la *view* da presentare.

Applicando tale struttura al Java Model 2:

- **Controller**: esegue la logica necessaria per ottenere e preparare i dati. Generalmente utilizza **Servlet** per coordinare l'elaborazione, imposta il contenuto tramite **Bean** e/o **POJO** e richiama le *view* appropriate (solitamente **JSP**).
- **View**: si occupa esclusivamente della renderizzazione del contenuto, mostrando i dati forniti dal controller senza includere logica applicativa.

11.3 Delega al container

Nell'architettura multi-tier molte delle funzionalità richieste dalle applicazioni (come gestione delle transazioni, sicurezza, pooling delle risorse, logging, ecc.) risultano comuni a più componenti dell'application server. Per evitare duplicazioni e semplificare lo sviluppo, tali responsabilità vengono delegate a un **container condiviso**.

Il container fornisce in modo trasparente una serie di servizi infrastrutturali ai componenti applicativi, tra cui:

- supporto al ciclo di vita dei componenti;
- supporto al sistema di nomi (naming e directory);
- supporto alla qualità del servizio (QoS);
- gestione della sicurezza;
- gestione delle risorse e delle transazioni.

11.3.1 Container proprietari

In questo approccio viene adottato un modello a componenti-container, in cui: i componenti implementano la logica applicativa mentre il container fornisce i servizi di base. Le modalità di interazione tra componenti e container sono definite, ma restano **proprietarie**, legate allo specifico application server e non interoperabili.

11.3.2 Container standard-based

Anche in questo caso viene utilizzato il modello componente-container, ma il container fornisce i servizi in conformità a **standard industriali condivisi**. Questo approccio garantisce portabilità dei componenti, interoperabilità tra piattaforme e riduce il *vendor lock-in*.

11.3.3 J2EE per applicazioni n-tier

La piattaforma J2EE supporta nativamente lo sviluppo di applicazioni distribuite secondo architetture a più livelli (*n-tier*), tra cui:

- **4-tier**: separazione tra client, web tier, business tier e data tier;
- **3-tier**: presentazione, logica di business e livello dati;
- **B2B**: architetture orientate all'integrazione tra sistemi aziendali differenti.

11.4 EJB

Gli **Enterprise JavaBeans (EJB)** rappresentano una tecnologia per lo sviluppo di applicazioni *server-side* in Java. Permettono di realizzare e distribuire applicazioni in modo semplificato, separando chiaramente la **logica di business** dal **codice infrastrutturale**.

EJB sfrutta un modello a componenti con **container pesante**, che fornisce servizi quali gestione delle transazioni, sicurezza, concorrenza e ciclo di vita. È previsto l'utilizzo dei *deployment descriptor* per la configurazione dei componenti.

Le applicazioni EJB e il server sono **debolmente accoppiati**: il comportamento dei componenti è definito tramite interfacce e non vi è una gestione esplicita delle risorse da parte del programmatore. Le applicazioni basate su EJB seguono naturalmente un'architettura *n-tier*.

11.4.1 Architettura

L'architettura EJB prevede un **container pesante** che opera all'interno di un *EJB server*. Il client interagisce con i componenti EJB in modo remoto attraverso interfacce ben definite, e tutte le invocazioni avvengono sempre tramite il container, che intercetta e gestisce i servizi infrastrutturali.

11.4.2 Descrittori

I **deployment descriptor** consentono di definire il comportamento dei componenti e le modalità di gestione del container, anche in fase di esecuzione (*runtime*).

Questi permettono:

- la configurazione dichiarativa dei servizi del container;
- la personalizzazione del comportamento dei componenti;
- una maggiore portabilità del codice.

A partire da Java 5, i deployment descriptor possono essere in gran parte **sostituiti dalle annotazioni**, riducendo la verbosità della configurazione.

11.4.3 Componenti

I principali tipi di componenti **EJB** sono:

- **Entity Bean** (Bean Managed Persistence e Container Managed Persistence): rappresentano una vista dei dati persistenti presenti all'interno di un database e, nella maggior parte dei casi, sono sincronizzati con esso. L'accesso ai dati è condiviso tra utenti differenti. Implementano l'interfaccia `javax.ejb.EntityBean`.
- **Session Bean**: sono tipicamente associati a un singolo client. Non sono persistenti e non rappresentano direttamente dati memorizzati in un database. Vengono utilizzati per modellare la logica di business relativa a un singolo utente e per coordinare le interazioni tra più bean.

I Session Bean possono essere:

- *Stateless*: eseguono una richiesta senza mantenere informazioni tra una chiamata e l'altra;
- *Stateful*: mantengono lo stato della conversazione con il client.

Implementano l'interfaccia `javax.ejb.SessionBean`.

- **Message-Driven Bean (MDB)**: basati su *Java Message Service (JMS)* e *Java API for XML Messaging*. Consumano messaggi in modo asincrono e non possono essere invocati direttamente dai client. Sono privi di stato e vengono attivati esclusivamente tramite l'inserimento di messaggi in una coda o in un topic.

Implementano l'interfaccia `javax.jms.MessageListener` e utilizzano il metodo `onMessage()` per la gestione della logica applicativa.

11.4.4 Bean in EJB 3.0

In EJB 3.0 i **Session Bean** e i **Message-Driven Bean** sono *POJO* (Plain Old Java Object), definiti e configurati principalmente attraverso l'uso di **annotazioni**, riducendo drasticamente la necessità dei deployment descriptor.

Gli **Entity Bean**, invece, vengono sostituiti dal modello di persistenza **JPA (Java Persistence API)**. In EJB 3.0 non esistono più Entity Bean come in EJB 2.x: la persistenza viene gestita tramite classi Java annotate come `@Entity`, che rappresentano le entità del dominio e sono mappate sulle tabelle del database. Questo approccio introduce numerosi vantaggi:

- semplificazione del modello di programmazione;
- maggiore leggibilità e manutenibilità del codice;
- riduzione dell'accoppiamento con il container;
- migliore integrazione con framework e librerie esterne.

11.5 Servizi container based

11.5.1 Gestione della concorrenza

È possibile gestire la concorrenza tra migliaia di client attraverso l'utilizzo di tecniche fornite direttamente dall'**EJB Container**, che solleva lo sviluppatore dalla gestione esplicita dei thread.

Le principali tecniche adottate sono:

- **Resource pooling**: nel caso di *Stateless Session Bean* e *Message-Driven Bean*, il container mantiene un insieme (pool) di istanze pronte all'uso. Le richieste dei client vengono servite assegnando dinamicamente un'istanza libera del pool (*instance pooling*). In modo analogo, il container può gestire anche il pooling delle risorse esterne, come connessioni a database o connector.
- **Activation / Passivation**: nel caso di *Stateful Session Bean*, il container può temporaneamente *passivare* un'istanza (serializzandone lo stato su memoria secondaria) quando non è in uso, per poi *attivarla* nuovamente quando il client ne ha bisogno. Questo meccanismo permette di ridurre il consumo di memoria e migliorare la scalabilità del sistema.

11.5.2 Stateless Session Bean

Ogni EJB Container mantiene un insieme di istanze del bean pronte all'uso. Non esiste uno stato di sessione associato al client: ogni invocazione è **indipendente** dalle precedenti e non conserva informazioni tra una chiamata e l'altra.

Per questo motivo viene adottata la tecnica dell'**instance pooling**, che consente di servire un elevato numero di richieste concorrenti in modo efficiente.

Il ciclo di vita di uno *Stateless Session Bean* può essere descritto attraverso i seguenti stati:

- **No state**: il bean non è ancora istanziato;
- **Pooled state**: il bean è istanziato ed inserito nel pool, ma non è associato ad alcuna richiesta utente;
- **Ready state**: il bean è temporaneamente associato ad una richiesta EJB ed è pronto a rispondere.

Ogni istanza del bean nel pool è associata ad un oggetto `javax.ejb.EJBContext`, che fornisce al bean un'interfaccia per interagire con il container.

Quando il bean si trova nello stato *Ready*, l'`EJBContext` contiene anche le informazioni relative al client corrente, oltre al riferimento allo *stub* EJB utilizzato per l'invocazione remota.

11.5.3 Stateful Session Bean

Gli *Stateful Session Bean* mantengono uno **stato conversazionale** associato ad uno specifico client per tutta la durata della sessione. A differenza degli *stateless bean*, ogni istanza è legata ad un singolo client.

La gestione degli *Stateful Session Bean* da parte del container si articola principalmente in due fasi:

- **Activation:** recupero dello stato del bean dalla memoria secondaria mediante *deserializzazione* e riassociazione dell'istanza al relativo oggetto EJB;
- **Passivation:** disassociazione temporanea del bean dal contesto di esecuzione e salvataggio del suo stato in memoria secondaria tramite *serializzazione*.

La procedura di *passivation* e *activation* viene utilizzata dal container per ottimizzare l'uso delle risorse, ad esempio quando il numero di client concorrenti è elevato.

È possibile intercettare questi cambiamenti di stato tramite specifiche annotazioni del ciclo di vita:

- `@javax.ejb.PrePassivate`: invocata immediatamente prima della passivazione del bean;
- `@javax.ejb.PostActivate`: invocata subito dopo la riattivazione del bean.

11.5.4 Message-Driven Bean

Analogamente agli *Stateless Session Bean*, i **Message-Driven Bean** (MDB) non mantengono alcuno stato di sessione associato al client. Per questo motivo il container può gestirne le istanze tramite meccanismi di *pooling* in modo semplice ed efficiente.

Le strategie di pooling adottate per i Message-Driven Bean sono simili a quelle utilizzate per gli *stateless session bean*. La principale differenza risiede nel fatto che un EJB container può mantenere **più pool distinti**, ciascuno associato ad una specifica *destination* JMS (coda o topic).

Ogni pool è quindi composto esclusivamente da istanze MDB configurate per consumare messaggi dalla stessa *destination*, consentendo al container di ottimizzare il consumo dei messaggi e la gestione della concorrenza.

11.6 Spring

Spring è un framework Java che implementa un modello a **container leggero**, progettato per semplificare lo sviluppo di applicazioni enterprise. A differenza dei container EJB tradizionali, Spring è non invasivo e si basa principalmente su POJO (*Plain Old Java Object*).

11.6.1 Funzionalità chiave

Le principali funzionalità offerte dal framework Spring sono:

- **Inversion of Control (IoC) e Dependency Injection (DI):** elimina il binding manuale tra i componenti dell'applicazione. Una `BeanFactory` (o `ApplicationContext`) si occupa della creazione, configurazione e gestione delle dipendenze tra gli oggetti, individuandoli tramite nome o tipo.
- **Supporto alla persistenza:** fornisce un livello di astrazione uniforme per la gestione delle transazioni e l'accesso ai database, integrandosi facilmente con framework di persistenza come JDBC, JPA e Hibernate.
- **Integrazione con il Web tier:** include un framework MVC (Spring MVC) per lo sviluppo di applicazioni web, basato sul pattern Model-View-Controller e fortemente integrato con il resto dell'ecosistema Spring.
- **Aspect-Oriented Programming (AOP):** permette la separazione delle *cross-cutting concerns* (come logging, sicurezza e gestione delle transazioni) dalla logica di business, migliorando modularità e manutenibilità del codice.

11.6.2 Architettura

Spring è un framework basato sul concetto di **micro-container** e su un modello a **componenti configurabili tramite proprietà**. È un framework **modulare**, organizzato in **layer** distinti e potenzialmente isolabili tra loro.

Spring permette la gestione centralizzata degli **oggetti di business**, semplificando il ciclo di vita dei componenti applicativi. Offre inoltre un'elevata facilità di **testing**, grazie al ridotto accoppiamento tra i componenti, e un'ottima capacità di **integrazione** con soluzioni e tecnologie preesistenti.

- **Core Package**: rappresenta il cuore del framework ed implementa i meccanismi di **Dependency Injection** e **Inversion of Control**. La presenza della `BeanFactory` (e dell'`ApplicationContext`) realizza il pattern *Factory* ed elimina la necessità di gestire manualmente i *singleton*, delegando tale responsabilità al container.
- **MVC Package**: fornisce il supporto allo sviluppo di applicazioni web secondo il pattern **Model-View-Controller**, permettendo una chiara separazione tra logica di business, controllo e presentazione.

11.6.3 Dependency Injection

Il container si occupa di risolvere le dipendenze tra gli oggetti configurando automaticamente le implementazioni richieste (*push* delle dipendenze). Questo approccio è opposto al pattern **Service Locator**, che opera secondo il principio di *pull*, in cui è l'oggetto stesso a dover richiedere esplicitamente le dipendenze.

I principali vantaggi della **Dependency Injection** sono:

- **Flessibilità**: il codice applicativo non contiene logica di lookup delle dipendenze;
- **Testabilità**: i componenti non dipendono direttamente dal container, rendendo possibile il testing automatico tramite oggetti mock o stub;
- **Manutenibilità**: il riutilizzo dei componenti è facilitato, poiché le dipendenze possono essere modificate agendo esclusivamente sulla configurazione.

La Dependency Injection può essere realizzata secondo due modalità principali:

- **Constructor Injection**: la dipendenza viene passata come parametro al costruttore dell'oggetto;
- **Setter Injection**: la dipendenza viene fornita tramite appositi metodi setter.

11.6.4 BeanFactory

L'oggetto `BeanFactory` è responsabile della creazione dei *bean* e della risoluzione delle loro dipendenze. Si tratta dell'interfaccia base del container Spring e rappresenta il meccanismo fondamentale su cui si basa la *Dependency Injection*.

Il container crea una `BeanFactory` utilizzando una specifica implementazione, come `XmlBeanFactory`, che legge la configurazione dei bean da un file XML contenente le informazioni necessarie all'injection.

`XmlBeanFactory` è un'estensione di `DefaultListableBeanFactory`.

```
public class XmlConfigWithBeanFactory {
    public static void main(String[] args) {
        XmlBeanFactory factory =
            new XmlBeanFactory(
                new FileSystemResource("beans.xml")
            );

        SomeBeanInterface b =
            (SomeBeanInterface) factory.getBean("nameOfTheBean");
    }
}
```

Configurazione XML dei bean Un esempio di configurazione tramite file `beans.xml` è il seguente:

```
<!DOCTYPE beans PUBLIC "-//SPRING//DTD BEAN//EN"
"http://www.springframework.org/dtd/spring-beans.dtd">

<beans>
    <bean id="renderer"
        class="StandardOutMessageRenderer">
        <property name="messageProvider">
            <ref local="provider"/>
        </property>
    </bean>

    <bean id="provider"
        class="HelloWorldMessageProvider"/>
</beans>
```

In questo caso la dipendenza viene iniettata tramite **setter injection**, utilizzando l'elemento `property`.

Injection tramite costruttore È possibile configurare le dipendenze anche a livello di costruttore, utilizzando il tag `constructor-arg`:

```
<beans>
    <bean id="provider"
        class="ConfigurableMessageProvider">
        <constructor-arg>
            <value>
                Questo è il messaggio configurabile
            </value>
        </constructor-arg>
    </bean>
</beans>
```

In questo caso Spring utilizza la **constructor injection** per istanziare il bean, passando i parametri direttamente al costruttore.

11.6.5 Parametri di injection

Spring supporta diverse tipologie di parametri per la *Dependency Injection*, consentendo una configurazione flessibile e dichiarativa delle dipendenze tra i componenti.

Le principali categorie di parametri di injection sono:

- **Valori semplici:** tipi primitivi e oggetti semplici come `String`, `int`, `boolean`, ecc.;
- **Bean definiti nella stessa factory:** riferimenti ad altri bean gestiti dal container;
- **Bean definiti in factory esterne**
- **Collezioni:** liste, set, mappe e proprietà;
- **Proprietà definite internamente:** valori o strutture dichiarate direttamente nel bean senza riferimenti esterni.

Per definire un riferimento a un altro bean gestito dal container si utilizza il tag `<ref>`, che permette di collegare un bean a un altro tramite il suo identificatore.

```
<bean id="service" class="MyService">
    <property name="repository">
        <ref bean="repositoryBean"/>
    </property>
```

</bean>

<bean id="repositoryBean" class="MyRepository"/>

In questo esempio il bean `service` riceve come dipendenza il bean `repositoryBean`, risolto automaticamente dal container Spring.

11.6.6 Naming dei componenti

Ogni bean deve avere un nome univoco all'interno della `BeanFactory`, poiché tale nome viene utilizzato dal container per l'identificazione e la risoluzione delle dipendenze.

Il nome del bean può essere definito, in ordine di priorità:

- tramite l'attributo `id`, che identifica in modo univoco il bean;
- tramite l'attributo `name`, che può contenere uno o più alias separati da virgola;
- in assenza di una definizione esplicita, dal nome della classe, generato automaticamente dal container secondo convenzioni predefinite.

Nel caso di utilizzo delle annotazioni, il nome del bean può essere specificato attraverso annotazioni come `@Component`, `@Service`, `@Repository` o `@Controller`. Se non viene fornito un nome esplicito, Spring assegna automaticamente un identificatore basato sul nome della classe, convertendo la prima lettera in minuscolo.

11.6.7 Supporto a MVC

Spring offre un supporto completo ai controller per lo sviluppo del modello MVC. In particolare, il controller gestisce le richieste dell'utente, prepara i dati (*model*) e determina la modalità con cui questi devono essere rappresentati (*view*), tipicamente tramite un meccanismo di *forward* o *redirect*.

Le principali caratteristiche del supporto MVC di Spring sono:

- chiara separazione dei ruoli tra *Model*, *View* e *Controller*;
- elevata adattabilità e riutilizzabilità dei componenti;
- flessibilità nel trasferimento del *model* tra controller e view;
- facilità di configurazione e integrazione con altri moduli del framework;
- meccanismi di *handler mapping* e *view resolution*;
- libreria di tag JSP per la semplificazione dello sviluppo delle view;
- componenti con ciclo di vita *scoped* (request, session, application).

11.6.8 Dispatcher servlet

La `DispatcherServlet` è una servlet centrale che si occupa di effettuare il *dispatch* delle richieste HTTP verso i controller appropriati. Il modello è completamente integrato nel concetto di *Inversion of Control* (IoC) ed è realizzato secondo il pattern **Front Controller**.

La `DispatcherServlet` intercetta tutte le richieste in ingresso, le analizza e, tramite i meccanismi di *handler mapping*, individua il controller responsabile della gestione della richiesta.

Come qualsiasi altra servlet, la `DispatcherServlet` necessita di un mapping all'interno del file `web.xml`, dove viene definita l'associazione tra gli URL e la servlet stessa.

Le azioni svolte dalla `DispatcherServlet` sono:

- associa un `WebApplicationContext` alla richiesta;
- associa un `LocaleResolver` alla richiesta;
- associa un `ThemeResolver` alla richiesta;
- ricerca un *handler*; se trovato, viene creata una catena di esecuzione composta da handler e interceptor;
- se il controller restituisce un *Model*, questo viene associato alla *View* appropriata per il rendering della risposta.

11.6.9 Controller

Spring supporta la creazione di controller secondo un modello astratto. L'interfaccia di base è rappresentata dalla classe **AbstractController**, che fornisce funzionalità di supporto come il *caching* e richiede l'override del metodo **handleRequestInternal**.

Questa classe può operare in modo autonomo e consente la gestione del caching a livello locale per l'utente, demandando al container la gestione degli aspetti infrastrutturali.

Altri modelli di controller messi a disposizione da Spring sono:

- **ParameterizableViewController**, utilizzato per mappare richieste direttamente a una *view* senza logica applicativa;
- **URLFilenameViewController**, che associa automaticamente una richiesta a una *view* sulla base del nome dell'URL.

CommandController Permettono il mapping automatico di una **HttpServletRequest** verso oggetti definiti (detti *command object*), generalmente JavaBean, popolati automaticamente con i parametri della richiesta.

I principali controller di questo tipo sono:

- **AbstractCommandController**: consente di definire il comportamento da applicare a un oggetto *command* (JavaBean) popolato automaticamente con i parametri della richiesta HTTP.
- **AbstractFormController**: offre supporto alla gestione dei form. Dopo la compilazione del form da parte dell'utente, i campi vengono inseriti nell'oggetto *command*. Il programmatore deve specificare i metodi per determinare quali *view* utilizzare per la presentazione del form e per la gestione della sottomissione.
- **SimpleFormController**: estende le funzionalità di **AbstractFormController** permettendo di specificare: il nome della *view* per il form e il nome della *view* da mostrare all'utente al completamento corretto della sottomissione.
- **AbstractWizardFormController**: fornisce supporto per la gestione di form multi-pagina (wizard). Il programmatore deve implementare i metodi astratti **validatePage()**, **processFinish()** e **processCancel()**. Altri metodi sovrascrivibili includono: **referenceData()** (per passare il model alla view come oggetto Map), **getTargetPage()** (per modificare dinamicamente l'ordine o il numero delle pagine del wizard), **onBindAndValidate()** (per personalizzare il processo di binding e validazione dei dati del form).

11.6.10 Handler

La funzionalità di base degli **handler** in Spring è fornita dalla classe **HandlerExecutionChain**. Questa classe consente di costruire una catena di esecuzione composta da un *handler* principale e da una lista di *interceptor*, che possono intercettare la richiesta **prima** e **dopo** l'esecuzione dell'handler.

Gli interceptor permettono di eseguire logica trasversale, come logging, controlli di sicurezza o gestione delle transazioni, senza modificare l'implementazione dell'handler stesso.

La maggior parte degli handler in Spring estende la classe **AbstractHandlerMapping**, che espone le seguenti proprietà principali:

- **interceptors**: lista di *handler interceptor* associati alla catena di esecuzione;
- **defaultHandler**: handler da eseguire nel caso in cui non venga trovato alcun mapping corrispondente alla richiesta;
- **order**: definisce la priorità e l'ordine di esecuzione degli handler mapping.

11.6.11 View

In Spring, i componenti dedicati alla gestione della rappresentazione grafica sono i **view resolver**. Essi semplificano il rendering delle view senza vincolarsi a una tecnologia specifica (ad esempio JSP, Thymeleaf, FreeMarker, ecc.).

Le due interfacce principali sono:

- **ViewResolver**: effettua il mapping tra il nome logico della view e l'implementazione concreta della view.
- **View**: si occupa della preparazione e della gestione della richiesta verso la view, inclusa la scrittura del modello (model) nell'output.

I **ViewResolver** più comuni sono: **AbstractCachingViewResolver**, **XmlViewResolver**, **UrlBasedViewResolver**.

11.6.12 Tag

La **Spring Tag Library** offre una serie di tag per lo sviluppo di JSP con Spring MVC. Questi tag permettono, in particolare, di:

- Accedere all'oggetto **Command** associato al form;
- Interagire con i dati gestiti dal **Controller**;
- Semplificare la gestione di input e output dei form nelle pagine JSP.

11.6.13 Gestione delle eccezioni

Spring fornisce il meccanismo di **HandlerExceptionResolver** per gestire eccezioni inaspettate durante l'esecuzione di un **Controller**.

- Le eccezioni contengono informazioni sul **Controller** in esecuzione.
- Ogni implementazione di **HandlerExceptionResolver** restituisce un oggetto **ModelAndView**, indicando la view da renderizzare.
- **SimpleMappingExceptionResolver** permette di mappare il nome della classe dell'eccezione a una specifica view, semplificando la gestione centralizzata degli errori.

11.6.14 Convenzioni e configurazione di default

Spring MVC supporta il principio di **Convention over Configuration**, che consente di ridurre al minimo la configurazione esplicita.

In pratica, è possibile ottenere il mapping automatico di controller, handler e view seguendo delle convenzioni predefinite, senza dover definire ogni singolo dettaglio nei file di configurazione.

Esempio In Spring MVC, la classe **ControllerClassNameHandlerMapping** utilizza convenzioni per determinare automaticamente il mapping tra URL e istanze di controller.

Ad esempio:

```
public class ViewShoppingCartController implements Controller {
    public ModelAndView handleRequest(HttpServletRequest request,
                                     HttpServletResponse response) {
        // logica di gestione
    }
}
```

La configurazione dei bean può essere effettuata così:

```
<bean class="org.springframework.web.servlet.mvc.support.ControllerClassNameHandlerMapping"/>
<bean id="viewShoppingCart"
      class="x.y.z.ViewShoppingCartController">
    ...
</bean>
```

ControllerClassNameHandlerMapping trova tutti i bean controller definiti e rimuove automaticamente il suffisso "Controller" dal nome della classe per generare il mapping URL:

- **WelcomeController** si mappa automaticamente su URL **/welcome***
- **HomeController** si mappa automaticamente su URL **/home***
- **IndexController** si mappa automaticamente su URL **/index***

12 Node.js e HTTP/3

12.1 Node.js

Node.js è una piattaforma server-side per lo sviluppo di applicazioni in JavaScript. Si basa sul motore **V8** di Google Chrome, che consente la compilazione del codice JavaScript in codice macchina, garantendo elevate prestazioni.

Node.js è progettato per offrire **elevata scalabilità e concorrenza** senza l'utilizzo di thread o processi dedicati per ogni richiesta. Il suo modello di esecuzione è **event-driven** e **non bloccante**, anche per le operazioni di input/output (I/O), rendendolo particolarmente adatto alla gestione di un grande numero di connessioni simultanee.

12.1.1 Event loop

In Node.js viene introdotto il concetto di **event loop**, che gestisce l'esecuzione delle operazioni asincrone attraverso una singola pila di esecuzione (*call stack*).

Questo modello riduce drasticamente il costo di overhead dovuto al *context switching* tipico dei sistemi multi-thread, migliorando l'efficienza e la scalabilità dell'applicazione.

Node.js utilizza il framework **CommonJS** per la gestione dei moduli, che permette di organizzare il codice in file separati e di importare funzionalità tramite il meccanismo di **require**.

Thread-based	Asynchronous Event-driven
Blocca l'applicazione o le richieste	Non blocca l'applicazione
Utilizza listener e worker thread	Utilizza un singolo thread
Ogni richiesta può essere gestita da un thread dedicato	Un solo thread estrae ripetutamente eventi da una coda
Usa il modello incoming-request	Usa una coda di eventi da processare
Un server multithread può bloccare una richiesta che coinvolge più eventi	Salva lo stato e passa subito al prossimo evento in coda
Fa ampio uso di context switching	Nessuna contesa e nessun context switching
Utilizza ambienti multithreading con lock sulle richieste	Evita lock grazie al modello single-thread
Gestione I/O spesso bloccante o semi-bloccante	Usa I/O asincrono basato su callback
Spesso utilizza poll/select o I/O bloccante	Non utilizza poll/select né O_NONBLOCK

Thread vs Event-Driven

12.1.2 Operazioni I/O sincrone

Normalmente i thread attendono la ricezione dei dati dal disco o dalla rete per proseguire con la loro esecuzione. Nel caso di server tradizionali si utilizzano thread pool e un thread per connessione per sopperire a queste limitazioni il che però porta ad un forte overhead dovuto al context switching.

I/O in Node.js In Node.js viene adottato il modello di **I/O asincrono non bloccante**. Ogni funzione che interagisce con operazioni di I/O (file system, rete, database) non blocca il flusso di esecuzione principale, ma registra una **callback** (o una *promise*) che verrà invocata al completamento dell'operazione.

Questo comportamento è possibile grazie alla natura **event-driven** di JavaScript e all'**event loop** di Node.js, che coordina l'esecuzione delle operazioni asincrone.

Gli script Node.js in attesa di operazioni di I/O **non spreca risorse di calcolo**: quando il codice non legato all'I/O termina la sua esecuzione, esso viene automaticamente rimosso dallo stack di chiamata. Al completamento dell'operazione asincrona, l'evento associato viene inserito nella coda degli eventi e gestito dal thread principale.

Questo modello consente a Node.js di supportare un elevato numero di connessioni contemporanee con un ridotto consumo di risorse, senza ricorrere a thread o processi dedicati.

12.1.3 In a nutshell

- utilizzo dello stesso linguaggio JavaScript impiegato nei browser anche lato server;
- gestione degli eventi tramite una coda e un *event loop*;

- utilizzo di interfacce di *wrapping* per le operazioni di I/O fornite dal sistema operativo;
- uso di un sistema di moduli per l'organizzazione del codice, basato su **import** ed **export**.

12.1.4 Quando utilizzare Node.js

Node.js è particolarmente adatto allo sviluppo di applicazioni server-side orientate al networking e alla gestione di un elevato numero di connessioni concorrenti.

Mette a disposizione una serie di moduli *core* per l'accesso alle funzionalità fondamentali della piattaforma (filesystem, networking, HTTP, ecc.), oltre a un vasto ecosistema di moduli esterni che permettono di velocizzare e semplificare lo sviluppo delle applicazioni.

12.1.5 Server side e Client side

La possibilità di utilizzare JavaScript sia lato client che lato server introduce importanti differenze nello sviluppo delle applicazioni.

In particolare, nello sviluppo **client-side** è possibile accedere e manipolare il DOM, interagire con l'utente e gestire eventi legati all'interfaccia grafica. Tuttavia, l'accesso alle risorse del sistema è fortemente limitato per motivi di sicurezza.

Al contrario, nello sviluppo **server-side** non è disponibile il DOM, ma è possibile accedere alla memoria persistente, al filesystem e alle risorse di rete, permettendo l'elaborazione dei dati, la gestione delle richieste e la comunicazione con database e servizi esterni.

12.1.6 Moduli

Il core di Node.js è composto da un insieme ristretto di moduli che si concentrano sui protocolli e sui formati di uso più comuni, come la gestione del filesystem, della rete e delle operazioni di I/O.

Per estendere le funzionalità di base è possibile utilizzare il package manager **npm** (Node Package Manager). **npm** consente la distribuzione, la condivisione e la modularizzazione del codice JavaScript sotto forma di pacchetti riutilizzabili ed è preinstallato con Node.js.

Express è un modulo Node.js ampiamente utilizzato per lo sviluppo di applicazioni web. Le sue principali caratteristiche sono:

- elevate prestazioni;
- supporto a diversi motori di templating;
- rapida generazione e configurazione di progetti web.

12.1.7 Listener/Emitter Pattern

Si definisce **listener** la funzione da eseguire nel momento in cui viene emesso un evento, ed **emitter** il soggetto che segnala il verificarsi di un evento. L'emissione di un evento comporta l'invocazione di tutti i listener ad esso associati.

In Node.js, l'esecuzione dei listener è **sincrona e bloccante** e avviene nell'ordine in cui i listener sono stati registrati. Per questo motivo è importante che i listener eseguano operazioni rapide oppure delegano le operazioni più costose a meccanismi asincroni, così da non bloccare l'event loop.

12.1.8 Stream

In Node.js sono presenti moduli che permettono la creazione e il consumo di **stream**, ovvero flussi di dati gestiti in maniera incrementale. Gli stream sono particolarmente utili nello sviluppo di server e applicazioni di rete, poiché consentono di elaborare grandi quantità di dati senza doverle caricare interamente in memoria. Gli stream possono essere creati o modificati dinamicamente e vengono utilizzati per operazioni di I/O come lettura e scrittura su file, socket di rete e comunicazioni HTTP.

Gli stream più comuni sono:

- **Readable stream**: consentono la lettura sequenziale di dati;

- **Writable stream:** consentono la scrittura sequenziale di dati;
- **Duplex stream:** permettono sia la lettura che la scrittura;
- **Transform stream:** estendono i duplex stream trasformando i dati durante il passaggio (ad esempio compressione o encoding).

12.1.9 TCP Networking

Node.js mette a disposizione il modulo **net**, che fornisce un wrapping delle chiamate di sistema relative alla comunicazione TCP.

```
var net = require('net');

net.createServer(callback)
  .listen(4000);
```

Il codice di esempio crea una socket TCP in ascolto sulla porta 4000. Per ogni connessione TCP in ingresso viene invocata la funzione di callback, che gestisce la comunicazione con il client.

12.2 HTTP/3

HTTP/3 è l'evoluzione del protocollo HTTP ed è progettato per migliorare le prestazioni e l'affidabilità delle comunicazioni web, in particolare in presenza di reti instabili o con elevata latenza. A differenza di HTTP/1.1 e HTTP/2, che si basano sul protocollo di trasporto TCP, HTTP/3 utilizza **QUIC**, un protocollo sviluppato inizialmente da Google e basato su **UDP**.

L'utilizzo di QUIC consente di superare alcune limitazioni di TCP, come il *head-of-line blocking*, migliorando la gestione delle connessioni multiple e riducendo i tempi di attesa.

Le principali caratteristiche di HTTP/3 sono:

- utilizzo di **QUIC** come protocollo di trasporto;
- riduzione della latenza grazie all'eliminazione del handshake TCP tradizionale;
- gestione indipendente dei flussi, evitando il blocco di una richiesta a causa di un'altra;
- maggiore resilienza ai cambiamenti di rete, come il passaggio tra reti Wi-Fi e mobili;
- sicurezza integrata, poiché la cifratura è nativa e basata su TLS 1.3.

HTTP/3 è particolarmente indicato per applicazioni web moderne che richiedono alte prestazioni, bassa latenza e una gestione efficiente di connessioni concorrenti, come servizi di streaming, applicazioni real-time e sistemi distribuiti.

13 Websocket e JSF

13.1 JSF

JavaServer Faces (JSF) è un framework Java per lo sviluppo di applicazioni web basate su un modello a componenti. Storicamente è stato costruito *on top* delle JSP, ma nelle versioni moderne utilizza **Facelets** come tecnologia principale per la definizione delle view.

JSF adotta un approccio **component-based**, in cui le pagine web sono costruite come alberi di componenti riutilizzabili. Tali componenti possono essere associati a oggetti server-side, detti **Managed Bean** o **backing bean**, che incapsulano la logica applicativa.

Il framework fornisce un ricco insieme di API per la definizione, la gestione e la rappresentazione dei componenti dell'interfaccia utente, oltre a meccanismi integrati per la gestione dello stato delle view e del ciclo di vita delle richieste.

JSF mette inoltre a disposizione un'ampia libreria di tag che consente di definire le interfacce in modo dichiarativo, favorendo una chiara separazione tra logica di presentazione e logica di business.

13.1.1 Utilizzo

Per definire un **Managed Bean** è sufficiente utilizzare l'annotazione `@ManagedBean`, che registra automaticamente il componente all'interno del container JSF. Una volta definito, il bean può essere utilizzato all'interno delle pagine XHTML tramite le **Expression Language (EL)**, ad esempio `#{hello.world}`.

Facelets rappresenta il linguaggio di definizione delle view in JSF e consente la costruzione dell'albero dei componenti dell'interfaccia utente. Facelets si integra con una servlet centrale, chiamata **FacesServlet**, responsabile dell'elaborazione delle richieste JSF e dell'invocazione del ciclo di vita del framework.

Come qualsiasi altra servlet, anche la **FacesServlet** necessita di un mapping all'interno del file `web.xml` (oppure tramite configurazione annotata nelle versioni più recenti).

13.1.2 Ciclo di vita

Il ciclo di vita di una richiesta **JSF** è suddiviso in una serie di fasi ben definite, gestite automaticamente dal framework.

- **Restore View**: alla ricezione di una richiesta viene creato o ripristinato l'albero dei componenti associato alla view.
- **Apply Request Values**: i valori inviati dal client vengono assegnati ai componenti dell'albero.
- **Process Validations**: vengono eseguite conversioni di tipo e validazioni automatiche sui dati di input.
- **Update Model Values**: i valori validati vengono copiati nei Managed Bean.
- **Invoke Application**: vengono eseguiti gli event handler e i metodi *action* associati ai componenti.
- **Render Response**: viene costruita e renderizzata la view finale inviata al client.

Durante il ciclo di vita, JSF gestisce automaticamente lo stato dei componenti, salvandolo e ripristinandolo in caso di richieste successive.

13.1.3 Templating

JSF supporta il concetto di **templating** tramite *Facelets*, consentendo la definizione di layout riutilizzabili da cui derivare più pagine.

Un template definisce una struttura comune dell'interfaccia, mentre le pagine derivate specificano esclusivamente i contenuti variabili.

I principali tag utilizzati per il templating sono:

- **ui:insert**: definisce una sezione del template riempibile dalle pagine derivate;
- **ui:define**: specifica il contenuto da inserire in una sezione;
- **ui:composition**: indica che la pagina utilizza un template;
- **ui:include**: include una pagina Facelet all'interno di un'altra;
- **ui:param**: consente il passaggio di parametri a template o componenti inclusi.

13.1.4 faces-config.xml

JSF utilizza un file di configurazione dedicato denominato **faces-config.xml**, che consente di definire in modo dichiarativo diversi aspetti dell'applicazione, in particolare:

- **Managed Bean**;
- **Navigation rules**.

Navigation rules Le *navigation rules* permettono di definire il flusso di navigazione dell'applicazione, modellabile come un grafo con molteplici ingressi e uscite. La navigazione avviene tramite *forwarding* verso una view di destinazione, specificata mediante il tag `<to-view-id>`.

Il meccanismo di navigazione si basa sul valore di ritorno (*outcome*) di un metodo *action* di un Managed Bean. In base all'outcome restituito, JSF seleziona la view successiva da visualizzare.

```
<navigation-case>
  <from-action>#{LoginForm.login}</from-action>
  <from-outcome>success</from-outcome>
  <to-view-id>/storefront.xhtml</to-view-id>
</navigation-case>
```

```
<navigation-case>
  <from-action>#{LoginForm.logon}</from-action>
  <from-outcome>failure</from-outcome>
  <to-view-id>/logon.xhtml</to-view-id>
</navigation-case>
```

Managed Beans I **Managed Bean** rappresentano i componenti server-side di JSF e incapsulano la logica applicativa associata alle view.

Sono semplici **JavaBean** che espongono proprietà e metodi detti *action*, invocati automaticamente in risposta agli eventi dell'interfaccia utente, in modo analogo alle classi *Action* del framework Struts.

I principali scope dei Managed Bean sono:

- **Application**: il bean vive per l'intera durata dell'applicazione;
- **Session**: il bean è condiviso all'interno di una sessione utente;
- **Request**: il bean vive per una singola richiesta HTTP;
- **View**: il bean vive finché la view è attiva (utile con AJAX).

I Managed Bean possono essere definiti tramite **faces-config.xml**:

```
<managed-bean-scope>request</managed-bean-scope>
```

oppure tramite annotazioni:

```
@ManagedBean
@RequestScoped
```

13.2 WebSocket

Il protocollo HTTP presenta una serie di limiti strutturali per la comunicazione **bidirezionale** (2-way) tra client e server. HTTP, infatti, nasce come protocollo **request/response** e non supporta nativamente comunicazioni persistenti e full-duplex.

Per aggirare tali limitazioni, nel tempo sono state introdotte diverse tecniche, tra cui:

- **Polling**: il client invia richieste al server a intervalli prefissati; il server risponde immediatamente, anche in assenza di nuovi dati, causando spreco di risorse e latenze artificiali.

- **Long polling:** il client invia una richiesta che rimane aperta finché il server non ha dati da inviare; dopo la risposta, il client apre una nuova connessione. Riduce il numero di richieste inutili, ma ogni risposta comporta comunque una nuova connessione.
- **Streaming / forever response:** il client invia una richiesta iniziale e il server mantiene la connessione aperta inviando dati in modo incrementale (*push*). Questa soluzione è solo **half-duplex** e presenta problemi di compatibilità e gestione delle risorse.
- **Connessioni multiple:** utilizzo di più connessioni HTTP parallele per simulare una comunicazione bidirezionale, con notevole overhead.

La tecnologia **WebSocket** nasce per superare questi limiti e fornire un meccanismo di comunicazione **full-duplex**, persistente ed efficiente tra client e server.

Il protocollo **WebSocket** è standardizzato nell’RFC 6455 (dopo alcune implementazioni proprietarie iniziali, come quelle sperimentali di Google Chrome) e consente di stabilire una connessione persistente su una singola socket TCP, sulla quale client e server possono inviare messaggi in modo asincrono e bidirezionale.

WebSocket è integrato nello standard **HTML5** ed è accessibile tramite JavaScript nei browser moderni. È inoltre supportato anche lato server, ad esempio in ambienti **Java EE**, Node.js e altri framework backend.

13.2.1 Caratteristiche

Le principali caratteristiche della tecnologia **WebSocket** sono:

- **bidirezionale:** client e server possono inviare messaggi in qualsiasi momento;
- **full-duplex:** comunicazione simultanea in entrambe le direzioni;
- **connessione unica e persistente:** viene utilizzata una sola connessione TCP per l’intera durata della comunicazione;
- **upgrade del protocollo HTTP:** la connessione WebSocket viene inizialmente stabilita tramite una richiesta HTTP e successivamente convertita (*upgrade*) in WebSocket;
- **uso efficiente di banda e CPU:** ridotto overhead rispetto alle tecniche basate su polling o long polling.

13.2.2 Elementi base

Nel modello WebSocket, il **client** inizializza la comunicazione inviando una richiesta HTTP contenente una richiesta di *upgrade* del protocollo. Il **server**, se supporta WebSocket, risponde accettando l’upgrade tramite un meccanismo detto **handshake**.

Una volta stabilita la connessione, entrambi gli endpoint vengono notificati dell’apertura del canale di comunicazione e possono:

- inviare messaggi in modo asincrono e bidirezionale;
- ricevere messaggi in qualsiasi momento;
- chiudere esplicitamente la connessione.

Ottimizzazione I messaggi WebSocket vengono trasmessi con un **overhead minimo**, grazie a header di dimensioni ridotte rispetto a HTTP. Il protocollo prevede inoltre la possibilità di **frammentare i messaggi** in più frame, consentendo l’invio efficiente di grandi quantità di dati senza bloccare la comunicazione.

13.2.3 Java API

Lato server, le **WebSocket API** sono definite dalla specifica **JSR-356**. Queste API permettono di sviluppare endpoint WebSocket utilizzando annotazioni, semplificando la gestione della comunicazione e del ciclo di vita.

Le principali funzionalità offerte sono:

- Gestione del **ciclo di vita** della connessione (@OnOpen, @OnClose, @OnError);
- Comunicazione tramite **messaggi** (@OnMessage);
- Possibilità di accedere alla **sessione** dell’endpoint;

- **Encoder** e **Decoder** per serializzazione e deserializzazione automatica di messaggi in oggetti Java.

Gli endpoint possono inviare e ricevere messaggi di tipo **testo** o **binary**.

Invio dei messaggi È possibile ottenere la sessione associata all'endpoint e da questa estrarre un `RemoteEndpoint`. Il `RemoteEndpoint` può essere:

- **Basic**: invio sincrono;
- **Async**: invio asincrono, non bloccante.

È inoltre possibile associare un singolo endpoint a più client connessi, ad esempio per realizzare una chat.

Ricezione dei messaggi I messaggi vengono ricevuti tramite metodi annotati con `@OnMessage`. Possono essere gestiti tre tipi principali di messaggi:

- **Testo**: messaggi stringa inviati dal client;
- **Binary**: messaggi binari (array di byte);
- **Pong**: risposta automatica a messaggi *ping*.

Stato Lato server, per ogni nuova connessione WebSocket viene creata un'istanza dell'endpoint. Questo permette di utilizzare **variabili di istanza** per memorizzare dati specifici della connessione o dell'utente associato.

È inoltre possibile accedere alle proprietà dell'utente tramite una **Map modificabile** fornita dalla sessione. Per dati condivisi tra tutte le connessioni o tutti gli utenti, possono essere utilizzate **variabili statiche** della classe.

Encoder Gli **encoder** in WebSocket servono a trasformare oggetti Java in messaggi che possono essere trasmessi attraverso la connessione, sia in formato testo che binario.

Per implementare un encoder è necessario estendere le classi generiche messe a disposizione dalla JSR-356:

- `Encoder.Text<T>`: per convertire un oggetto Java in una stringa da inviare;
- `Encoder.Binary<T>`: per convertire un oggetto Java in un array di byte.

Una volta implementato, l'encoder può essere associato a un endpoint tramite annotazioni specifiche. Ogni volta che il server o il client invia un oggetto con `sendObject`, il container WebSocket:

1. verifica il tipo dell'oggetto da inviare;
2. seleziona l'encoder corrispondente registrato per quel tipo;
3. esegue la conversione dell'oggetto in messaggio testo o binario;
4. invia il messaggio al peer connesso.

È importante notare che ogni istanza di encoder è legata a una specifica connessione e a un singolo peer WebSocket. Questo significa che eventuali variabili di stato all'interno dell'encoder possono essere usate per gestire dati relativi alla singola connessione, senza interferire con altre sessioni.

Decoder I **decoder** hanno il compito opposto: ricevono messaggi in formato testo o binario e li trasformano in oggetti Java utilizzabili dal server o dal client.

Per implementarli si utilizzano le interfacce fornite dalla JSR-356:

- `Decoder.Text<T>`: trasforma messaggi di tipo stringa in oggetti Java;
- `Decoder.Binary<T>`: trasforma messaggi binari in oggetti Java.

Anche i decoder vengono associati agli endpoint tramite annotazioni, e il container si occupa di:

1. intercettare il messaggio ricevuto;
2. selezionare il decoder appropriato in base al tipo del messaggio;

3. invocare il metodo di decoding per ottenere l'oggetto Java;
4. passare l'oggetto decodificato all'endpoint per l'elaborazione.

In questo modo, encoder e decoder permettono di gestire facilmente la conversione tra oggetti Java e messaggi WebSocket, astrando completamente il formato di trasmissione e semplificando lo sviluppo di applicazioni bidirezionali complesse.

13.2.4 Integrazione con JavaScript

Lato client, le WebSocket possono essere utilizzate direttamente in JavaScript tramite il costruttore:

```
var socket = new WebSocket(url[, protocols]);
```

dove `url` è l'indirizzo del server WebSocket e `protocols` è opzionale, per specificare uno o più sottoprotocollo(i) da negoziare con il server.

Le principali proprietà disponibili sull'oggetto `WebSocket` sono:

- **bufferedAmount**: quantità di dati (in byte) ancora presenti nel buffer in attesa di essere inviati;
- **onclose**: funzione callback invocata quando la connessione viene chiusa;
- **onerror**: funzione callback invocata in caso di errore nella connessione o nella trasmissione;
- **onmessage**: funzione callback invocata quando viene ricevuto un messaggio dal server;
- **onopen**: funzione callback invocata quando la connessione viene stabilita correttamente;
- **protocol**: sottoprotocollo selezionato dal server durante l'handshake;
- **readyState**: stato attuale della connessione (0 = CONNECTING, 1 = OPEN, 2 = CLOSING, 3 = CLOSED);
- **url**: URL a cui il client è connesso.

I principali metodi disponibili sono:

- **send(data)**: invia dati (testo, `ArrayBuffer` o `Blob`) al server;
- **close([code, reason])**: chiude la connessione, opzionalmente specificando un codice di chiusura e una motivazione.

Gli eventi fondamentali per la gestione di una connessione WebSocket lato client sono:

- **open**: attivato quando la connessione con il server viene stabilita correttamente, utile per inizializzare operazioni come l'invio di messaggi;
- **message**: attivato ogni volta che il client riceve un messaggio dal server. L'evento contiene la proprietà `data` con il contenuto ricevuto;
- **error**: attivato in caso di errore nella connessione o durante la trasmissione dei dati, utile per gestire problemi di rete o protocollari;
- **close**: attivato quando la connessione viene chiusa, sia dal client che dal server. Fornisce informazioni come il codice di chiusura e la motivazione.